Semantic correctness and noninterference of code extraction for structural type theory with erasure

Constantine Theocharis, Edwin Brady

arxiv:2605.00655v1; CC BY4.0 (paper); MIT (separate fixed author formal proof)

Editorial note. Full native author paper is independently CC BY4.0; complete fixed formal continuation is separately MIT with original LICENSE.

Editorial note. Only one code extraction correctness construction counted; native paper conservativity and other support are not separate tasks.

Editorial note. The author states the formal model internally in Psh(lambda); original postulates/type-in-type assumptions remain printed and source preserved. No Agda execution or proof certification.

Editorial note. README TODOs concern separate zeroing/conservativity files. They are retained as original disclosure, not promoted to solved proofs.

Editorial note. Native old-arrows CTAN font/style assets include separate LPPL1.3-or-later/SIL OFL1.1 notices; mathpartir GPL2-or-later and quiver MIT are explicitly bundled. Local DejaVu mono font retains its license.

Editorial note. fvextra1.14.0 official full source/README (LPPL1.3-or-later) bundled; package code mechanically extracted from dtx package guards without TeX execution. Actual required etoolbox/fancyvrb/upquote/lineno/textcomp/kvoptions/keyval/ltxcmds/kvs runtime files and their original notices are explicit assets; full upstream source archives of first four retained. No shared TeX changes.

Editorial note. All actual root-log missing glyphs (U2113,U+FF1D,U+FF20,U+1D7D9) have font-only fancyvrb active-character mappings: DejaVuSans for script ell/double-struck1; original DroidSansFallbackFull for fullwidth equals/@. Characters and raw Agda files remain unchanged; both fallback fonts and respective original licenses are explicit assets.

Editorial scope. Complete author material collection for one structural theorem; author retains responsibility for the mathematics.

Allowed standard prerequisites (editorial).

- SOGAT initial syntax and displayed-model/gluing metatheory. Original paper section2 and references to Uemura/Bocquet/Kaposi retained in full author article; no recursive formal verification.
- Author internal presheaf interpretation of target lambda calculus. Original paper footnote and Models/FO/ExtractionCorrectness.agda initial

comments and target-interface postulates retained verbatim. Author uses type-in-type/observational equality and beta/eta/natural-number target interfaces; these are disclosed original assumptions.

1 Introduction

When types depend on values, it becomes unclear which parts of a program are needed at runtime. One can perform whole-program analysis steps to heuristically detect and remove the majority of data that is not computationally relevant [10, 44]. However, the unbounded abstraction capabilities of dependent types make such techniques brittle and unpredictable for more complicated examples. What seems to have become the standard way to erase runtime-irrelevant data is to enrich the underlying type theory with a separate sort for terms to be erased by compilation. One popular approach is quantitative type theory (QTT) [34, 7]. Parameterised by a *quantitative semiring*, it encapsulates not only erasure but also various forms of substructural variable usage, including linearity. When instantiated to the ordered semiring $\{0 < \omega\}$, the resulting theory could be described as 'Martin-Löf type theory (MLTT) with erasure'. This approach is implemented by Agda [3] and Idris [25]. Besides this, there is a mechanism of 'ghost sorts' [46] planned for Rocq [47] that achieves similar goals.

We contribute yet another approach to erasure, in the style of algebraic, reduction-free 'type theory in type theory' [5]. The main difference over previous work is that our formulation is fully structural, corresponding to a well-studied class of type theories identified by Uemura [45]. As a result, we can rely on certain aspects of its metatheory and its implementation that are already understood in the general setting. We present erasure as a modular *second-order generalised algebraic theory* (SOGAT) extending MLTT, compatible with cumulative universes and inductive families.

From this formulation, we derive both syntactic and semantic results. On the syntactic side, we establish conservativity over MLTT (theorem 17 and corollary 18) and the independence of runtime data from erased data (theorem 14). The former ensures that erasure does not alter the proving power of type theory, while the latter is our main theorem of 'well-behavedness'.

On the semantic side, we extend the standard interpretation of type theory in terms of sets and functions to account for erasure (definition 20). This is possible when there is a tiny proposition available in the interpretation category. We give a fully worked example in the category of families of sets and discuss how it extends to Grothendieck toposes.

We then construct a code extraction model (definition 21) producing untyped lambda calculus terms that only retain runtime data, along with a correctness proof by gluing that extracted code always tracks the standard interpretation (theorem 27). This is intended to be implemented as part of the compilation pipeline post-typechecking.

We provide a demo implementation elaborating a high-level language with erased binders similar to Idris or Agda into the core language presented here (section 8), and extracting untyped lambda calculus terms in the end. We include supplementary notes on the implementation of pattern unification for

metavariables. Our theoretical results are formalised in Agda; the symbol ↺ appearing throughout the paper is a hyperlink to the relevant part of the formalisation.

2 An informal presentation of erasure

Here we present erasure informally through a high-level dependently typed language to be elaborated into a yet unspecified core syntax. This behaves essentially the same as Idris/Agda with `0`/`@0` annotations. For now, we use type-in-type for simplicity.

Modes and usages Terms are split into the *runtime* mode (ω) and the *erased* mode (0):

- runtime terms are written as $a \overset{\omega}{:} A$
- erased terms are written as $a \overset{0}{:} A$
- every runtime term can be used as an erased term

We order these as $0 < \omega$. This asymmetry reflects that at compile-time everything is accessible but at runtime only non-erased data is. This is a different kind of compile-time/runtime distinction from program staging or metaprogramming. A compile-time term will not necessarily have a *known value* at compile-time; rather, it will not survive *past* the compilation phase. The runtime phase comes after the compilation phase, so any data that reaches that point is also accessible before it – at compile-time. Variables are also annotated with a mode $\{0, \omega\}$. We will generally write $a : A$ for $a \overset{\omega}{:} A$, explicitly indicating only erased terms. In the core language the subusaging rule will be implemented by an explicit coercion mechanism (section 2.1).

Functions We write $(x \overset{i}{:} A) \to B$ for the type of dependent functions from A to B where the domain is at mode $i \in \{0, \omega\}$. A function with a runtime domain $(x : A) \to B$ survives compilation, while a function with an erased domain $(x \overset{0}{:} A) \to B$ is compiled to its return value directly. In the latter case, the rules of the theory ensure that the return value will not have any dependency on the input x that can be observed at runtime. For example, this results in every closed function $(n \overset{0}{:} \mathsf{Nat}) \to \mathsf{Nat}$ being compiled to a constant numeral (theorem 27).

Universes The universe of types, denoted U, is not computationally relevant: we only need an erased code for a type to decode it. In other words, if $A \overset{0}{:} \mathsf{U}$ then A is a type. For type dependency, such as in $(x \overset{i}{:} A) \to B$, regardless of the mode i of the bound variable, it suffices for B to be applied to an erased $x \overset{0}{:} A$ to yield a type. This means there is an equivalence between type families $(x \overset{0}{:} A) \to \mathsf{U}$ and $(x \overset{\omega}{:} A) \to \mathsf{U}$.

Example 1. The identity function $\lambda\{A\}\,x.\ x : \{A \overset{0}{:} \mathsf{U}\} \to A \to A$ extracts to the untyped identity function $\lambda x.\ x$, erasing the type argument A. Here we freely use the notation $\{x \overset{i}{:} A\} \to B$ for an implicit function type, which is merely an elaboration feature.

Example 2. The signature describing length-indexed lists, or vectors, is

$$\begin{aligned}
&\mathsf{Vec} : (A \overset{0}{:} \mathsf{U}) \to (n \overset{0}{:} \mathsf{Nat}) \to \mathsf{U}\\
&\mathsf{nil} : \{A \overset{0}{:} \mathsf{U}\} \to \mathsf{Vec}\ A\ \mathsf{zero}\\
&\mathsf{cons} : \{A \overset{0}{:} \mathsf{U}\} \to \{n \overset{0}{:} \mathsf{Nat}\} \to (x : A) \to \mathsf{Vec}\ A\ n \to \mathsf{Vec}\ A\ (\mathsf{succ}\ n)
\end{aligned}$$

This is a standard example of erasure, where vectors do not store their length at runtime due to the usage of 0 in the n argument of cons. The types are also all erased.

Pairs Similar to functions, $(x \overset{i}{:} A) \times B$ is the type of mode-aware dependent pairs; if $i = 0$ then the first projection is erased, and if $i = \omega$ it exists at runtime. We can also extend this to customise the mode of the second projection $(x \overset{i}{:} A) \times^j B$, which is equivalent to $(x \overset{i}{:} A) \times ((_ \overset{j}{:} B) \times \mathsf{Unit})$ but with a better runtime representation in the case where $j = 0$.

Example 3. Erased dependent pairs capture the situation where we have some index that needs to be packaged existentially but which we never access at runtime. For example, we can package the erased length of a vector, to get a list:

$$\begin{aligned}
&\mathsf{List}\ A : \mathsf{U}\\
&\mathsf{List}\ A = (n \overset{0}{:} \mathsf{Nat}) \times \mathsf{Vec}\ A\ n
\end{aligned}$$

At runtime, only the second projection of an erased pair survives, so if defined this way, lists and vectors have the same runtime representation.

Example 4. If the second projection is erased instead, this captures the situation where we want to carry a proof about a runtime object that shouldn't exist at runtime. We can define the type of natural numbers less than n by

$$\begin{aligned}
&\mathsf{Fin} : \mathsf{Nat} \to \mathsf{U}\\
&\mathsf{Fin}\ n = (k : \mathsf{Nat}) \times_0 \mathsf{Lt}\ k\ n
\end{aligned}$$

for an appropriately defined less-than predicate $\mathsf{Lt} : (k : \mathsf{Nat}) \to (n : \mathsf{Nat}) \to \mathsf{U}$.

Inductive types Inductive types are carried over from MLTT; the signature in example 2 is a valid such instance. The only new rule is that the mode of the scrutinee in eliminators must be *at least as strong* as the mode of the output. In other words, we cannot pattern match on compile-time data at runtime. Besides that, there is a new axis of choices from the ability to mark parts of inductive types as erased. We can mark data arguments or recursive arguments as erased, or even entire constructors. This is explored in section 4.

Example 5. For booleans, given $x \overset{i}{:} \mathsf{Bool}$ and $a,\ b \overset{j}{:} A$ where $i \geq j$ we have

$$\mathsf{if}\ x\ \mathsf{then}\ a\ \mathsf{else}\ b \overset{j}{:} A\,.$$

This means that if we are in erased mode, we can perform case analysis on any boolean, but if we are in runtime mode, we can only perform case analysis on a runtime boolean.

2.1 Erasure as a phase distinction

When we transition from the surface language to the core language, the sub-usaging rule of erasure is explained through a *synthetic phase distinction*. The concept of 'phase distinction' originates from Cardelli [11], later reformulated synthetically by Sterling and Harper [42].

A synthetic phase distinction is simply an *abstract* proposition #, meaning a propositional sort of our type theory to which we do not attach a specific truth value. We are allowed to bind proofs of # in contexts, but there is no way to produce a closed proof of #, or store proofs of # inside terms or types. If a context Γ contains the proposition #, we write $\# \in \Gamma$, which itself is a *decidable* proposition in the syntax.

Example 6. Using $\triangleright$ for context extension and $\bullet$ for the empty context, we have:

$$\# \in (\bullet \triangleright x \overset{0}{:} \mathsf{Nat} \triangleright \# \triangleright y \overset{\omega}{:} \mathsf{Fin}\ x) \qquad \# \notin (\bullet \triangleright x \overset{0}{:} \mathsf{Nat} \triangleright y \overset{\omega}{:} \mathsf{Fin}\ x)$$

In total, we have three kinds of context extension: extending by a runtime term variable, extending by an erased term variable, or extending by a # proof variable.

We call the propositional sort # the *erasure marker*, because a context is considered erased when $\# \in \Gamma$. In such an erased context Γ, runtime and erased terms become interchangeable through two new constructors $\uparrow$ and $\downarrow$. The precise rules surrounding these are:

- If $a \overset{0}{:} A$ and $\# \in \Gamma$, then $\uparrow a \overset{\omega}{:} A$.
- If $a \overset{\omega}{:} A$ in context $\Gamma \triangleright \#$, then $\downarrow a \overset{0}{:} A$ in context Γ.
- $\uparrow$ and $\downarrow$ are mutual inverses up to definitional equality.

To provide an erased term, it suffices to provide a runtime term under the assumption of # – a uniqueness property similar to 'every function is a lambda' and 'every product is a pair'. Crucially, # cannot be introduced or discharged in any other way (it is not a type!). The justification for the first point above is that $\# \in \Gamma$ holds only when we are already in a *subterm* of an erased term. This way, we emulate the *phase distinction requirement* of Cardelli that every subterm of a compile-time term should itself be compile-time.

This mechanism allows us to treat *any* runtime term as erased, eliminating the need to separately axiomatise the erased variant of a term when the runtime variant is already present in the theory. The decidability of $\# \in \Gamma$ means we can build an elaboration algorithm which inserts all $(\uparrow, \downarrow)$ coercions (section 8). The high-level syntax we have presented thus far is the input to this elaboration.

3 Formal setup

In this section we formally develop type theory with erasure, assuming familiarity with category theory and in particular categories with families (CwFs) [13]. We work in a constructive intensional type theory with a bounded cumulative hierarchy of universes $\mathbf{Set}_\ell$ ($\ell \leq \omega + 1$), natural numbers $\mathbb{N}$, function extensionality, uniqueness of identity proofs, and quotient inductive-inductive types [4]. We write $\mathbf{Psh}_\ell(C)$ for the category of presheaves over C valued in $\mathbf{Set}_\ell$. In general, when we omit ℓ, we mean $\ell = \omega$. We also omit some equality transports for readability. When working internally to categories, we overload type-theoretic notation; $(x : A) \to B$ might denote a function type in a presheaf category [22] or in some theory of signatures (section 3). We use $\mathcal{U}$ for internal universes, mentioning this explicitly when necessary. We write TT_0 for type theory with erasure and TT for ordinary Martin-Löf type theory. Rather than fixing type formers, we develop the theory modularly. For example, 'TT with Π-types' denotes MLTT with only function types.

Generalised algebraic theories A generalised algebraic theory (GAT) [12] is a description of a theory consisting of sorts, operations and equations which are allowed to appear in arbitrary order where dependency is permitted. Such a description can be neatly given by a context in a certain dependent type theory called the *GAT theory of signatures* [31]. This supports $\mathbb{1}$, Σ, equality types with UIP, a universe $\mathcal{U}$ for declaring sorts, and dependent function types Π with domain in $\mathcal{U}$, and Π_{ext} with an external type (e.g. $\mathbb{N}$) as domain. GATs can be used to describe type theories in an intrinsically well-formed fashion.

Example 7. The GAT Cat_1 of a category with a terminal object is given by:

Con	$: \mathcal{U}$	id	$: \mathsf{Sub}\ \Gamma\ \Gamma$
Sub	$: \mathsf{Con} \to \mathsf{Con} \to \mathcal{U}$	$- \circ -$	$: \mathsf{Sub}\ \Phi\ \Delta \to \mathsf{Sub}\ \Gamma\ \Phi \to \mathsf{Sub}\ \Gamma\ \Delta$
$\bullet$	$: \mathsf{Con}$	$\mathsf{id}\circ$	$: \mathsf{id} \circ \sigma = \sigma$
ϵ	$: \mathsf{Sub}\ \Gamma\ \bullet$	$\circ\mathsf{id}$	$: \sigma \circ \mathsf{id} = \sigma$
$\epsilon\eta$	$: (\sigma : \mathsf{Sub}\ \Gamma\ \bullet) \to \sigma = \epsilon$	assoc	$: (\sigma \circ \tau) \circ \rho = \sigma \circ (\tau \circ \rho)$

Definition 8 (Σ-CwF)**.** *A Σ-CwF is a CwF equipped with $\mathbb{1}$ and Σ types. A morphism of Σ-CwFs is a strict CwF morphism preserving $\mathbb{1}$ and Σ, inducing a category $\mathbf{CwF}_\Sigma \subseteq \mathbf{CwF}$.*

Each GAT G gives rise to a *freely generated* Σ-CwF, where base types are determined by the sorts of G, and base terms by the operations of G quotiented by the equations in G. This is the approach of Bocquet [8], which equips GATs with *functorial semantics* [32]. We identify G with its freely generated Σ-CwF. In this language, a model $\mathcal{M}$ of a GAT G is simply a Σ-CwF morphism $\mathcal{M} : G \to \mathbf{Set}$, with the standard Σ-CwF structure on $\mathbf{Set}$. To give a morphism out of G, it suffices to define its actions on the signature items of G.

Example 9. A morphism $\mathcal{F} : \mathsf{Cat}_1 \to E$ is determined by a closed type $\mathcal{F}.\mathsf{Con}$ in E and a type $\mathcal{F}.\mathsf{Sub}\ \Gamma\ \Delta$ over $\Gamma : \mathcal{F}.\mathsf{Con}$ and $\Delta : \mathcal{F}.\mathsf{Con}$ in E, such that the terminal object, identity and composition are preserved.

Second-order generalised algebraic theories Second-order generalised algebraic theories (SOGATs) [45, 8] are a generalisation of GATs which allow second-order binding to appear in signatures. The *SOGAT theory of signatures* extends the GAT theory of signatures with a subuniverse $\mathcal{U}_{\mathsf{R}} \subset \mathcal{U}$ and a dependent function type Π_{R} in $\mathcal{U}$ with domain in $\mathcal{U}_{\mathsf{R}}$.

Example 10. The simplest interesting SOGAT is untyped lambda calculus with $\beta\eta$ laws:

$$\begin{aligned} &\mathsf{Tm} : \mathcal{U}_{\mathsf{R}} \\ &(\mathsf{lam}, \mathsf{app}, \beta, \eta) : (\mathsf{Tm} \to \mathsf{Tm}) \simeq \mathsf{Tm} \end{aligned} \tag{1}$$

The isomorphism $(\mathsf{Tm} \to \mathsf{Tm}) \simeq \mathsf{Tm}$ is shorthand for two operations lam, app and two coherence equations β, η. The forward direction lam implicitly uses Π_{R} to bind the second-order occurrence of Tm, while the backward direction app uses the first-order Π.

SOGATs are an even more convenient tool to describe type theories because they already include a notion of variable binding. This avoids the boilerplate of encoding variables and substitution manually, at least for *structural* theories (ones that do not involve linearity, contextual modalities or other constraints on variable usage).

Definition 11 ($(\Sigma, \Pi_{\mathsf{R}})$-CwF)**.** *A* $(\Sigma, \Pi_{\mathsf{R}})$-CwF *is a* Σ*-CwF with a subpresheaf* $\mathsf{Ty}_{\mathsf{R}} \subseteq \mathsf{Ty}$ *of* representable types *closed under* $\mathbb{1}$ *and* Σ*, and a function type* Π_{R} *with domain in* Ty_{R}*, inducing a category* $\mathbf{CwF}_{\Sigma,\Pi_{\mathsf{R}}} \subseteq \mathbf{CwF}_{\Sigma}$*.*

Similar to before, a SOGAT can be viewed as a freely generated $(\Sigma, \Pi_{\mathsf{R}})$-CwF. This time, sorts in $\mathcal{U}_{\mathsf{R}}$ become the base types in Ty_{R}. A model $\mathcal{M}$ of a SOGAT S consists of a category $\mathcal{M}_\diamond$ with terminal object and a $(\Sigma, \Pi_{\mathsf{R}})$-CwF morphism $\mathcal{M} : S \to \mathbf{Psh}(\mathcal{M}_\diamond)$. The codomain $\mathbf{Psh}(\mathcal{M}_\diamond)$ has a standard $(\Sigma, \Pi_{\mathsf{R}})$-CwF structure where the representable types are presheaves with a 'context extension' operation on $\mathcal{M}_\diamond$. This includes the types of $\mathcal{M}_\diamond$ if $\mathcal{M}_\diamond$ itself is a CwF, by the Yoneda embedding y. The category $\mathcal{M}_\diamond$ should be thought of as the underlying *category of contexts* of $\mathcal{M}$, where second-order binding in S is interpreted using context extension along representable types in $\mathcal{M}_\diamond$. Once again, to give a morphism out of S it suffices to define its actions on the signature items of S.

Correspondence between GAT and SOGAT models Bocquet's approach to the model theory of SOGATs is to reduce them to GATs and then reuse the model theory of GATs. From a SOGAT S we can compute a GAT S^{fo} extending Cat_1, such that SOGAT models $\mathcal{M} : S \to \mathbf{Psh}(\mathcal{M}_\diamond)$ are in bijective correspondence with GAT models $\widetilde{\mathcal{M}} : S^{\mathsf{fo}} \to \mathbf{Set}$ where the Cat_1 part of S^{fo} maps to $\mathcal{M}_\diamond$. We often reuse the name $\mathcal{M}$ for $\widetilde{\mathcal{M}}$ as it is unambiguous to do so. The mapping $S \mapsto S^{\mathsf{fo}}$ essentially corresponds to the signature-based translation from SOGATs to GATs detailed by Kaposi and Xie [28]: it maps a SOGAT S to the GAT of a category with terminal object and the presheaf interpretation of S over it. This generates all the variable and substitution boilerplate machinery necessary to interpret the second-order binding in S in terms of first-order context extension. This mapping is *functorial*: a SOGAT morphism $S \to T$ yields a GAT morphism $S^{\mathsf{fo}} \to T^{\mathsf{fo}}$ that preserves the category structure.

Example 12. Translating the SOGAT of untyped lambda calculus from eq. (1) produces a GAT extending Cat_1 with a family $\mathsf{Tm} : \mathsf{Con} \to \mathcal{U}$ of terms with substitution $-[-] : \mathsf{Tm}\ \Delta \to \mathsf{Sub}\ \Gamma\ \Delta \to \mathsf{Tm}\ \Gamma$ preserving identity and composition (in other words, a presheaf), a context extension operator $-\triangleright : \mathsf{Con} \to \mathsf{Con}$ representing the presheaf Tm via $\mathsf{Sub}\ \Gamma\ (\Delta\triangleright) \simeq \mathsf{Sub}\ \Gamma\ \Delta \times \mathsf{Tm}\ \Gamma$, and an isomorphism $(\mathsf{lam}, \mathsf{app}) : \mathsf{Tm}\ (\Gamma\triangleright) \simeq \mathsf{Tm}\ \Gamma$ encoding the second-order constructor lam using context extension.

Categories of models For any two models $\mathcal{M}, \mathcal{N}$ of a GAT G, there is a function space model $\mathbf{Func}(\mathcal{M}, \mathcal{N})$ of natural transformations from $\mathcal{M}$ to $\mathcal{N}$, displayed over G. For example, a context displayed over Γ in $\mathsf{Func}(\mathcal{M}, \mathcal{N})$ is a function $\mathcal{M}\,\Gamma \to \mathcal{N}\,\Gamma$. A strict morphism of models $\mathcal{M} \to \mathcal{N}$ is thus defined as a dependent Σ-CwF morphism $G \to \mathbf{Func}(\mathcal{M}, \mathcal{N})$. This yields the usual notion of GAT morphism that can be computed from signatures [31], and induces a category of G-models $\mathbf{Mod}(G)$. The syntax $\mathbf{0}_G$ of a GAT G is the initial G-model, which always exists and is given by a quotient inductive-inductive type; for any other G-model $\mathcal{M}$, we write $[\![-]\!]_{\mathcal{M}} : \mathbf{0}_G \to \mathcal{M}$ for the unique G-model morphism. The syntax $\mathbf{0}_S$ of a SOGAT S is simply $\mathbf{0}_{S^{fo}}$. A strict morphism of GATs $\mathcal{F} : G \to H$ yields a functor between categories of models $\mathcal{F}^* : \mathbf{Mod}(H) \to \mathbf{Mod}(G)$ by precomposition. Applying this to $\mathbf{0}_H$, we get a G-model $\mathcal{F}^*\ \mathbf{0}_H$. We often abuse notation and write $\mathcal{F}$ instead of $[\![-]\!]_{\mathcal{F}^*\mathbf{0}_H}$.

4 TT_0 as a SOGAT

Now we proceed to define TT_0 as a SOGAT and compute its GAT translation. The full definition is given in fig. 1, with the extension to inductive types described in section 4.

$$\mathsf{Mode} \triangleq \{0, \omega\}$$
$$\mathsf{Ty} : \mathbb{N} \to \mathcal{U}$$
$$\mathsf{Tm} : \mathsf{Mode} \to \mathsf{Ty}_\ell \to \mathcal{U}_{\mathsf{R}}$$
$$\# : \mathcal{U}_{\mathsf{R}}$$
$$\#\text{-}\mathsf{prop} : (p, q : \#) \to p = q$$
$$(\downarrow, \uparrow) : (\# \to \mathsf{Tm}_\omega\ A) \simeq \mathsf{Tm}_0\ A$$
$$\Pi_i : (A : \mathsf{Ty}_\ell) \to (\mathsf{Tm}_i\ A \to \mathsf{Ty}_\ell) \to \mathsf{Ty}_\ell$$
$$(\mathsf{lam}, \mathsf{app}) : ((x : \mathsf{Tm}_i\ A) \to \mathsf{Tm}_\omega\ (B\ x)) \simeq \mathsf{Tm}_\omega\ (\Pi_i\ A\ B)$$
$$\Sigma_i : (A : \mathsf{Ty}_\ell) \to (\mathsf{Tm}_i\ A \to \mathsf{Ty}_\ell) \to \mathsf{Ty}_\ell$$
$$(\mathsf{pair}, \mathsf{proj}) : ((x : \mathsf{Tm}_i\ A) \times \mathsf{Tm}_\omega\ (B\ x)) \simeq \mathsf{Tm}_\omega\ (\Sigma_i\ A\ B)$$
$$\mathsf{U} : (\ell : \mathbb{N}) \to \mathsf{Ty}_{\ell+1}$$
$$(\mathsf{El}, \mathsf{code}) : \mathsf{Tm}_0\ \mathsf{U}_\ell \simeq \mathsf{Ty}_\ell$$

Figure 1: The SOGAT defining TT_0 with Π, Σ, and universes ($\mathcal{U}$).

TT_0 is a theory involving three sorts: types Ty externally indexed by universe levels in $\mathbb{N}$, terms Tm externally indexed by modes $\{0, \omega\}$ and internally indexed by types, and the *erasure marker* $\#$, forced to be a proposition by $\#$-prop. The definitional isomorphism $(\downarrow, \uparrow)$ converts between modes: if $a \overset{0}{:} A$ and $p : \#$, then $\uparrow_p a \overset{\omega}{:} A$; if $a \overset{\omega}{:} A$ under assumption $p : \#$, then $\downarrow_{p.} a \overset{0}{:} A$. In $\uparrow_p a$, p is an argument, while in $\downarrow_{p.} a$, p is *bound* in a, shorthand for $\downarrow(\lambda p.\ a)$. Since we can always weaken $a : \mathsf{Tm}_\omega\ A$ to $\lambda_.a : \# \to \mathsf{Tm}_\omega\ A$, we can convert ω

terms to 0 terms in any context. By induction on i, we implicitly extend this to $\downarrow : \mathsf{Tm}_i\ A \to \mathsf{Tm}_0\ A$.

We include Σ and Π types whose introduction and elimination rules involve terms in mode ω, while type formation binds a term of mode i matching the domain mode. Universes have codes *in mode 0*. This induces an isomorphism $(\# \to \mathsf{Ty}_\ell) \simeq \mathsf{Ty}_\ell$ which allows us to convert between type families $\mathsf{Tm}_\omega\ A \to \mathsf{Ty}_\ell$ and $\mathsf{Tm}_0\ A \to \mathsf{Ty}_\ell$. Without universes we would have to include this isomorphism as a primitive. This yields an alternative equivalent formulation of Π_i and Σ_i where the binder in the type formation rule is always indexed by $\mathsf{Tm}_0\ A$, for example: $\Sigma_i : (A : \mathsf{Ty}_\ell) \to (\mathsf{Tm}_0\ A \to \mathsf{Ty}_\ell) \to \mathsf{Ty}_\ell$. Also, it is possible and practically desirable to include strict cumulativity [39] for universes, which we omit here.

The erased fragment The theory only explicitly includes terms for the runtime fragment ω. Usually, presentations of theories with erasure will include a copy of terms in each mode. In our case, this is not necessary. By virtue of the isomorphism $(\downarrow, \uparrow)$, we automatically get the entire erased fragment 'for free'. For example, erased terms for function types at any domain mode are derivable:

$$\begin{aligned}
&\mathsf{lam}_0 : ((x : \mathsf{Tm}_0\ A) \to \mathsf{Tm}_0\ (B\ x)) \to \mathsf{Tm}_0\ (\Pi_i\ A\ (B \circ \downarrow)) \\
&\mathsf{lam}_0\ f = \downarrow_p \mathsf{lam}\ (\lambda x.\ \uparrow_p (f\ \downarrow x))
\end{aligned}$$

$$\begin{aligned}
&\mathsf{app}_0 : \mathsf{Tm}_0\ (\Pi_i\ A\ B) \to (x : \mathsf{Tm}_0\ A) \to \mathsf{Tm}_0\ ((B \circ \uparrow)\ x) \\
&\mathsf{app}_0\ t\ x = \downarrow_p \mathsf{app}\ (\uparrow_p t)\ (\uparrow_p x)
\end{aligned}$$

Here we use $- \circ \uparrow$ and $- \circ \downarrow$ for the two directions of $(\mathsf{Tm}_\omega\ A \to \mathsf{Ty}_\ell) \simeq (\mathsf{Tm}_0\ A \to \mathsf{Ty}_\ell)$. The isomorphism providing β and η for $(\mathsf{lam}, \mathsf{app})$ also holds for $(\mathsf{lam}_0, \mathsf{app}_0)$. The idea is that if the goal is of the form $\mathsf{Tm}_0\ X$, then we introduce $\downarrow_p$ and reduce the goal to $\mathsf{Tm}_\omega\ X$ while having access to p. Then we use whatever runtime term former to fulfil the goal. When such a runtime term former *expects* a $\mathsf{Tm}_\omega\ Y$ but all we have is a $\mathsf{Tm}_0\ Y$, we wrap it in $\uparrow_p$ using the p we bound earlier. We can derive the rest of the erased fragment this way. From now on, we use the subscript $_0$ as in lam_0 to denote the erased variant of some term former.

Inductive types It is straightforward to include any indexed inductive type in TT_0. We only need to include it in mode ω, and then we can derive its erased fragment using the technique in section 4. However, the main affordance of erasure is that we can choose for some constructor data, usually indices, to *always* be erased. For example, here is the inductive family of length-indexed

vectors, where the length is considered erased data:

$$
\begin{aligned}
&\mathsf{Vect} : \mathsf{Tm}_0\ \mathsf{Nat} \to \mathsf{Ty}_\ell \to \mathsf{Ty}_\ell\\
&\mathsf{nil} : \mathsf{Tm}_\omega\ (\mathsf{Vect}\ \mathsf{zero}\ A)\\
&\mathsf{cons} : \{k : \mathsf{Tm}_0\ \mathsf{Nat}\} \to \mathsf{Tm}_\omega\ A \to \mathsf{Tm}_\omega\ (\mathsf{Vect}\ k\ A) \to \mathsf{Tm}_\omega\ (\mathsf{Vect}\ (\mathsf{succ}_0\ k)\ A)\\
&\mathsf{elim} : (P : (n : \mathsf{Tm}_0\ \mathsf{Nat}) \to \mathsf{Tm}_0\ (\mathsf{Vect}\ n\ A) \to \mathsf{Ty}_\ell)\\
&\quad \to \mathsf{Tm}_\omega\ (P\ \mathsf{zero}_0\ \mathsf{nil}_0)\\
&\quad \to (\{k : \mathsf{Tm}_0\ \mathsf{Nat}\} \to (x : \mathsf{Tm}_\omega\ A) \to (xs : \mathsf{Tm}_\omega\ (\mathsf{Vect}\ k\ A))\\
&\qquad \to \mathsf{Tm}_\omega\ (P\ k\ xs) \to \mathsf{Tm}_\omega\ (P\ (\mathsf{succ}_0\ k)\ (\mathsf{cons}_0\ x\ xs)))\\
&\quad \to \{n : \mathsf{Tm}_0\ \mathsf{Nat}\} \to (v : \mathsf{Tm}_\omega\ (\mathsf{Vect}\ n\ A)) \to \mathsf{Tm}_\omega\ (P\ n\ v)
\end{aligned}
$$

We have omitted type indexing and computation rules for brevity. Erased constructor arguments that appear in return indices can sometimes be converted to *relevant* arguments. A sufficient condition for this is Brady et al.'s forcing analysis [10], which can be applied here: if we make the n in the output above *relevant*, we can also make k in the cons method relevant because k can be computed at runtime by stripping succ from n. It is possible to extend code extraction (definition 21) with such 'forced' eliminators, and other optimisations in [10] like detagging.

We could include general W-types [24] in TT_0, but this would not afford us the most generality. This is because we have a choice of erased data both in the non-recursive and the recursive arguments. The former would be possible with standard W-types because we have mode-dependent Σ types, but the latter wouldn't. For example, consider the type

$$
\begin{aligned}
&\mathsf{Nat0} : \mathsf{Ty}_\ell\\
&\mathsf{zero0} : \mathsf{Tm}_\omega\ \mathsf{Nat0}\\
&\mathsf{succ0} : \mathsf{Tm}_0\ \mathsf{Nat0} \to \mathsf{Tm}_\omega\ \mathsf{Nat0}
\end{aligned}
$$

This cannot be encoded as a regular W-type because W-types encode *arities* of recursive arguments, but now there is also the axis of $0/\omega$ to choose from. Additionally, we might want entire constructors to only exist in mode 0:

$$
\begin{aligned}
&\mathsf{Bool}_{\omega\to\mathsf{true}} : \mathsf{Ty}_\ell\\
&\mathsf{true} : \mathsf{Tm}_\omega\ \mathsf{Bool}_{\omega\to\mathsf{true}}\\
&\mathsf{false} : \mathsf{Tm}_0\ \mathsf{Bool}_{\omega\to\mathsf{true}}
\end{aligned}
$$

This is the type of booleans that are always true at runtime. So there is also an axis of $0/\omega$ for the return sort of each constructor. Altogether these 'exotic' inductive types suggest that it would be useful to investigate *signatures* for inductive types with erasure to better understand this space of possibilities. We leave this to future work.

Compatibility with two-level type theory Having formulated erasure as a SOGAT, we can now easily combine it with other features formulated as SOGATs. For example, we can form a theory $\mathsf{2LTT}_0$ of two-level type theory [30] with erasure. Aside from its use in HoTT [6], two-level type theory can be viewed as a type theoretic formulation of staging/metaprogramming. It allows us to

control which binders are evaluated at compile-time versus runtime, existing in the same space of features as erasure – features that aim to improve the feasibility of dependent types for practical programming.

The general principle of two-level type theory is that some or all judgments in the object theory become type formers in the meta-theory. In this case, our object theory is type theory with erasure. Therefore, we promote Tm_ω and Tm_0 from sorts to meta-level types (but not #, to retain the decidability of its presence in any context):

$$\begin{aligned} &\mathsf{Ty}^{\mathsf{M}} : \mathcal{U} && \Uparrow : (i : \mathsf{Mode}) \to \mathsf{Ty} \to \mathsf{Ty}^{\mathsf{M}} \\ &\mathsf{Tm}^{\mathsf{M}} : \mathsf{Ty}^{\mathsf{M}} \to \mathcal{U}_{\mathsf{R}} && (\langle -\rangle, \sim -) : \mathsf{Tm}^{\mathsf{M}}\ \Uparrow_i A \simeq \mathsf{Tm}_i\ A \end{aligned}$$

Here the superscript M denotes the meta fragment, while the base theory corresponds to the object fragment. In the meta fragment, we have two separate lifting operations $\Uparrow_i$, one for each mode i. The erasure marker # and coercions $\downarrow / \uparrow$ remain unchanged from before. From them we can derive coercions in the meta level as well:

$$(\downarrow^{\mathsf{M}}, \uparrow^{\mathsf{M}}) : (\# \to \mathsf{Tm}^{\mathsf{M}}\ \Uparrow_\omega A) \simeq \mathsf{Tm}^{\mathsf{M}}\ \Uparrow_0 A$$

These are given by $\downarrow^{\mathsf{M}} f = \langle \downarrow_{p.}\ \sim(f\ p)\rangle$ and $\uparrow^{\mathsf{M}}_p x = \langle \uparrow_p\ \sim x\rangle$.

4.1 Generated first-order theory ($\mathcal{TT}$)

Using the results from section 3, we now compute the GAT specification of TT_0. This is the actual 'type system' in the traditional sense, which includes contexts and variables. As with any SOGAT, we start with a category with terminal object described in example 7.

Sorts For each sort in the SOGAT, we get a presheaf on this category:

$$\mathsf{Ty} : \mathbb{N} \to \mathsf{Con} \to \mathcal{U} \qquad \mathsf{Tm} : \mathsf{Mode} \to (\Gamma : \mathsf{Con}) \to \mathsf{Ty}_\ell\ \Gamma \to \mathcal{U} \qquad \# \in - : \mathsf{Con} \to \mathcal{U}$$

There is also an operation $(x, y : \# \in \Gamma) \to x = y$ from #-prop that forces $\# \in -$ to be proposition-valued. We now see that $\# \in \Gamma$ (section 2.1) is simply the presheaf generated by the sort # evaluated at a context Γ. Each of these presheaves comes with a substitution operation $A[\sigma]$, $t[\sigma]$, $\pi[\sigma]$ which respects $\circ$ and id.

Context extensions The representability of mode-indexed terms and the erasure marker yields the context extension operations

$$- \triangleright_i - : (\Gamma : \mathsf{Con}) \to \mathsf{Ty}_\ell\ \Gamma \to \mathsf{Con} \qquad - \triangleright \# : \mathsf{Con} \to \mathsf{Con}$$

with corresponding isomorphisms

$$\begin{aligned} &((-,_i -), \mathsf{pq}_i) && : (\sigma : \mathsf{Sub}\ \Gamma\ \Delta) \times \mathsf{Tm}_i\ \Gamma\ A[\sigma] \simeq \mathsf{Sub}\ \Gamma\ (\Delta \triangleright_i A) \\ &((-,_\# -), \mathsf{pq}_\#) && : \mathsf{Sub}\ \Gamma\ \Delta \times (\# \in \Gamma) \simeq \mathsf{Sub}\ \Gamma\ (\Delta \triangleright \#) \end{aligned}$$

defining that substitutions can be extended by terms and erasure marker witnesses, and that terms in both modes as well as the erasure marker have the

'0th' de Bruijn index by q and weakening by p. More explicitly, we get

$$\begin{array}{llll} \mathsf{p}_i & : \mathsf{Sub}\ (\Gamma \rhd_i A)\ \Gamma & \mathsf{p}_\# & : \mathsf{Sub}\ (\Gamma \rhd \#)\ \Gamma \\ \mathsf{q}_i & : \mathsf{Tm}_i\ (\Gamma \rhd_i A)\ A[\mathsf{p}_i] & \mathsf{q}_\# & : \# \in (\Gamma \rhd \#) \end{array}$$

which are derivable from the isomorphisms above, satisfying the usual CwF rules.

Erasure coercions The defining isomorphism of # is presented in the GAT as

$$(\downarrow, \uparrow) : \mathsf{Tm}_\omega\ (\Gamma \rhd \#)\ A \simeq \mathsf{Tm}_0\ \Gamma\ A$$

relating runtime terms in a context extended by # to erased terms. The $\uparrow$ direction outputs a term in an extended context. We can also derive a form which stores the data of the substitution needed to make the context general:

$$\uparrow' : \# \in \Gamma \to \mathsf{Tm}_0\ \Gamma\ A \to \mathsf{Tm}_\omega\ \Gamma\ A \qquad \uparrow_p{}' t = (\uparrow t)[\mathsf{id}\ ,_\#\ \mathsf{p}_\#]$$

Just like before, we can extend $\downarrow$ to operate on a term in any mode i – now explicitly:

$$\downarrow_* : \mathsf{Tm}_i\ \Gamma\ A \to \mathsf{Tm}_0\ \Gamma\ A \qquad \downarrow_*\ \{i = 0\}\ t = t \qquad \downarrow_*\ \{i = \omega\}\ t = \downarrow (t[\mathsf{p}_\#])$$

Standard types The rest of the theory is translated to a first-order representation that looks very similar to the usual CwF structures. For Π types we get

$$\begin{array}{l} \Pi_i : (A : \mathsf{Ty}_\ell\ \Gamma) \to (B : \mathsf{Ty}_\ell\ (\Gamma \rhd_i A)) \to \mathsf{Ty}_\ell\ \Gamma \\ (\mathsf{lam}, \mathsf{app}) : \mathsf{Tm}_\omega\ (\Gamma \rhd_i A)\ B \simeq \mathsf{Tm}_\omega\ \Gamma\ (\Pi_i\ A\ B) \end{array}$$

matching the usual CwF formulation: the codomain B lives over the mode-i extension $\Gamma \rhd_i A$. We can write the less 'categorical' application operator as

$$\mathsf{app}' : \mathsf{Tm}_\omega\ \Gamma\ (\Pi_i\ A\ B) \to (x : \mathsf{Tm}_i\ \Gamma\ A) \to \mathsf{Tm}_\omega\ \Gamma\ B[\langle x \rangle]$$

where $\langle - \rangle \triangleq (\mathsf{id},_i -) : \mathsf{Tm}_i\ \Gamma\ A \to \mathsf{Sub}\ \Gamma\ (\Gamma \rhd_i A)$ is the single term substitution. The derivable erased fragment (section 4) is also carried over to the first-order GAT presentation.

5 Syntactic properties

In this section, we explore some properties of the first-order syntax of TT_0.

Zeroing The first property we show is that erased terms do not depend on runtime variables. In particular, an erased term in Γ should uniquely correspond to an erased term in 0Γ, which is Γ with the modes of all the variables set to 0, and without any erasure markers #. This verifies that the phase distinction truly prevents erased data from depending on runtime data. We do this by showing that the erased fragment of TT_0 is an appropriate model for the entire theory (the *zeroing* model), such that zeroing erased TT_0 terms is a bijection.

Definition 13 (Zeroing 𝒪). *The* zeroing $(\Sigma, \Pi_{\mathsf{R}})$*-CwF endomorphism* $0 : \mathsf{TT}_0 \to \mathsf{TT}_0$ *uses the erased fragment of* TT_0 *to implement the relevant fragment. It sets the mode of all terms to erased, and removes any erasure markers* $\#$*. It is defined by its action on the signature components:*

$$\begin{array}{lll} 0.\mathsf{Ty}_\ell & \triangleq \mathsf{Ty}_\ell & 0.\mathsf{Tm}_0\ A \triangleq \mathsf{Tm}_0\ A \\ 0.\# & \triangleq \mathbb{1} & 0.\mathsf{Tm}_\omega\ A \triangleq \mathsf{Tm}_0\ A \end{array}$$

The rest of the signature is implemented by the erased fragment (section 4). For example $0.\Pi_i\ A\ B \triangleq \Pi_i\ A\ B$*,* $0.\mathsf{lam}\ f \triangleq \mathsf{lam}_0\ f$ *and* $0.\mathsf{app}\ f\ x \triangleq \mathsf{app}_0\ f\ x$*. The* $\uparrow/\downarrow$ *become trivial.*

By acting on the syntax, this yields a morphism of $\mathsf{TT}_0^{\mathsf{fo}}$ models $[\![-]\!]_0 : \mathbf{0}_{\mathsf{TT}_0} \to 0^*\ \mathbf{0}_{\mathsf{TT}_0}$. We can compute the actions on syntactic sorts as $0_{\mathsf{Con}} : \mathsf{Con} \to \mathsf{Con}$, $0_{\mathsf{Ty}} : \mathsf{Ty}\ \Gamma \to \mathsf{Ty}\ 0\Gamma$, $0_{\mathsf{Tm}_\omega} : \mathsf{Tm}_\omega\ \Gamma\ A \to \mathsf{Tm}_0\ 0\Gamma\ 0A$, $0_{\mathsf{Tm}_0} : \mathsf{Tm}_0\ \Gamma\ A \to \mathsf{Tm}_0\ 0\Gamma\ 0A$, and $0_\# : \# \in \Gamma \to \mathbb{1}$.

Theorem 14 (Types and erased terms need nothing 𝒪). *In the syntax of* TT_0 *there exists a substitution* $\Uparrow : \mathsf{Sub}\ \Gamma\ 0\Gamma$ *which becomes the identity under zeroing, such that* $(0A)[\Uparrow] = A$ *for types and* $(0a)[\Uparrow] = \downarrow_\star a$ *for terms. This induces natural isomorphisms:*

$$\mathsf{Ty}_\ell\ 0\Gamma \simeq \mathsf{Ty}_\ell\ \Gamma \qquad\qquad \mathsf{Tm}_0\ 0\Gamma\ 0A \simeq \mathsf{Tm}_0\ \Gamma\ A$$

Conservativity over type theory We can also characterise the relationship between TT_0 and TT. In particular, we would like to ensure that erasure does not allow the production of any exotic terms that are not possible in ordinary type theory. The formal notion of this condition is *conservativity*: if there exists a proof of a TT-theorem in TT_0, then a proof also exists in TT. This can be shown by constructing bidirectional interpretations.

Definition 15 (TT_0 to TT 𝒪). *The morphism* $\llcorner - \lrcorner : \mathsf{TT}_0 \to \mathsf{TT}$ *is constructed by using* TT *types and terms to implement* TT_0 *types and terms (both runtime and erased). In this model, mode annotations and the erasure marker are forgotten:*

$$\begin{array}{lll} \llcorner - \lrcorner.\mathsf{Ty}_\ell & \triangleq \mathsf{Ty}_\ell & \llcorner - \lrcorner.\mathsf{Tm}_0\ A \triangleq \mathsf{Tm}\ A \\ \llcorner - \lrcorner.\# & \triangleq \mathbb{1} & \llcorner - \lrcorner.\mathsf{Tm}_\omega\ A \triangleq \mathsf{Tm}\ A \end{array}$$

Definition 16 (TT to TT_0 𝒪). *Conversely, the morphism* $\ulcorner - \urcorner : \mathsf{TT} \to \mathsf{TT}_0$ *is constructed by using the erased fragment to implement the structure of* TT*:*

$$\ulcorner - \urcorner.\mathsf{Ty}_\ell \triangleq \mathsf{Ty}_\ell \qquad\qquad \ulcorner - \urcorner.\mathsf{Tm}\ A \triangleq \mathsf{Tm}_0\ A$$

The composition $\ulcorner \llcorner - \lrcorner \urcorner$ is almost equivalent to zeroing 0 (but doesn't preserve Π and Σ modes), while $\llcorner \ulcorner - \urcorner \lrcorner$ is the identity. In the terminology of Bocquet [8, p. 40], $\llcorner - \lrcorner$ is a *trivial fibration*, yielding the following properties:

Theorem 17 (Erased conservativity of TT_0 over TT 𝒪). TT_0*'s erased fragment is conservative over* TT*: there is a surjective natural map* $\mathbf{0}_{\mathsf{TT}_0}.\mathsf{Tm}_0\ \ulcorner\Gamma\urcorner\ \ulcorner A\urcorner \to \mathbf{0}_{\mathsf{TT}}.\mathsf{Tm}\ \Gamma\ A$.

Corollary 18 (Runtime conservativity of TT_0 over TT 🐢)**.** *TT_0's runtime fragment is conservative over TT: given a TT_0 context Γ' and type A' such that $0\Gamma' = \ulcorner \Gamma \urcorner$ and $0A' = \ulcorner A \urcorner$, there is a natural map $\mathbf{0}_{\mathsf{TT}_0}.\mathsf{Tm}_\omega\ \Gamma'\ A' \to \mathbf{0}_{\mathsf{TT}}.\mathsf{Tm}\ \Gamma\ A$.*

We might be tempted to ask for a stronger conservativity result for erased terms, namely that this map is actually an isomorphism. This is not possible if we have mode-aware binder types. For example, consider for each mode i the term $\mathsf{pair}\ (\mathsf{code}\ (\Pi_i\ \mathsf{Nat}\ \mathsf{Nat}))\ (\mathsf{lam}_0\ \mathsf{q}_0) : \mathsf{Tm}_0 \bullet (\Sigma_0\ \mathsf{U}\ (\mathsf{El}\ \mathsf{q}_0))$. For either choice of i we get the same TT term, so $\llcorner - \lrcorner$ is not injective even on erased terms.

6 Standard models

The standard semantics of Martin-Löf type theory are in **Set**, and can be generalised to presheaf categories [22] or Grothendieck toposes [18]. Now we explore standard models for TT_0, first at the level of sets, and then briefly at the generality of Grothendieck toposes. The utility of such models, besides showing that the theory can be modelled by known and common mathematical objects, lies in their use when proving metatheorems via gluing [27].

The involvement of a phase distinction in TT_0 requires the standard semantics to take place not just in plain sets, but rather in some kind of *phase-separated* sets. The isomorphism $(\# \to \mathsf{Tm}_\omega\ A) \simeq \mathsf{Tm}_0\ A$ shows us that in the semantics we must have an object $\#$ such that exponentiating by $\#$ *progresses* the phase from runtime to erased. It is already known that languages with phase separations have semantics in glued categories [40]. A glued category is a comma category of the form $\mathsf{Id}_C \downarrow F$ for a functor $F : D \to C$. In the simplest case, we take $C = D = \mathbf{Set}$ and $F = \mathsf{Id}_{\mathbf{Set}}$. This yields the arrow category of **Set**, which is the category of presheaves over the interval $0 \to 1$, and also equivalent to the category of families of sets $\mathsf{Fam}(\mathbf{Set})$. We choose to work with $\mathsf{Fam}(\mathbf{Set})$, using its internal language when convenient.

The category $\mathsf{Fam}(\mathbf{Set})$ serves as the simplest standard model of TT_0, which we denote by $\mathcal{S}$. For a family $(X_0 : \mathbf{Set}) \times (X_1 : X_0 \to \mathbf{Set})$, the base X_0 stores the erased data, and the fibers store the runtime data. Consider the object $\phi \triangleq (\mathbb{1}, \lambda_.\ \mathbb{0})$, the family with a single empty fiber. It is a proposition in the sense that any two maps into ϕ are equal. Exponentiation of $X = (X_0, X_1)$ by ϕ acts as $(\phi \to X) \simeq (X_0,\ \lambda_.\ \mathbb{1})$, so maps out of ϕ isolate the base component, trivialising the fibers. This suggests that the erasure marker $\#$ should be interpreted as ϕ. Because we are now in a semantic setting, we can interpret erased terms directly as maps from ϕ to runtime terms: $\mathcal{S}.\mathsf{Tm}_0\ A = (\phi \to \mathcal{S}.\mathsf{Tm}_\omega\ A)$.

Definition 19 (P-modal)**.** *Given a proposition P, an object X is P-*modal *if the weakening map $\mathsf{p} : X \to (P \to X)$ defined by $\mathsf{p}\ x\ _ = x$ is an isomorphism.*

In $\mathsf{Fam}(\mathbf{Set})$, an object is ϕ-modal if its fibers are contractible. A consequence of this approach is that any part of the language that is erased should be ϕ-modal. This notably includes universes. The standard Hofmann-Streicher [23] universe construction can be performed in $\mathsf{Fam}(\mathbf{Set})$, yielding the universe $\mathcal{U}_\ell \triangleq (\mathbf{Set}_\ell, \lambda A.\ A \to \mathbf{Set}_\ell)$ for $\ell < \omega$. However, this is not ϕ-modal, since its fibers are not contractible. We need an alternative universe construction. Let

us suggestively denote this by $\sqrt[\phi]{\mathcal{U}_\ell}$ at level ℓ. We will revisit this notation in section 6. For the $\mathsf{El}/\mathsf{code}$ isomorphism of TT_0, we have:

$$\mathcal{S}.\mathsf{U}_\ell \triangleq \sqrt[\phi]{\mathcal{U}_\ell} \qquad \mathcal{S}.\mathsf{code}\ (A : \sqrt[\phi]{\mathcal{U}_\ell}) \triangleq \mathsf{p}\ A : \phi \to \sqrt[\phi]{\mathcal{U}_\ell} \qquad \mathcal{S}.\mathsf{El}\ (c : \phi \to \sqrt[\phi]{\mathcal{U}_\ell}) \triangleq \boxed{?} : \sqrt[\phi]{\mathcal{U}_\ell}$$

An assignment to $\boxed{?}$ such that El and code form an isomorphism is possible when $\sqrt[\phi]{\mathcal{U}_\ell}$ is ϕ-modal. Luckily, $\mathsf{Fam}(\mathbf{Set})$ supports a universe which is ϕ-modal:

$$\sqrt[\phi]{\mathcal{U}_\ell} \triangleq ((A : \mathbf{Set}_\ell) \times (A \to \mathbf{Set}_\ell), \lambda_.\mathbb{1})$$

The decoding map of this universe is of the form $[-] : \sqrt[\phi]{\mathcal{U}_\ell} \to \mathcal{U}_\ell$, and is defined by first projection for the base, and second projection for the fibers. It supports all base types of $\mathcal{U}_\ell$, and is closed under dependent products and sums. We call this universe *squashed* because it is ϕ-modal but still retains all the original structure. Now we are ready to define the full model:

Definition 20 ($\mathsf{Fam}(\mathbf{Set})$ model of TT_0 🕜**).** *The standard model of* TT_0 *in families of sets is* $\mathcal{S} : \mathsf{TT}_0 \to \mathbf{Psh}_{\omega+1}(\mathsf{Fam}(\mathbf{Set}))$*, given by:*

$$\begin{array}{lll} \mathcal{S}.\mathsf{Ty}_\ell & \triangleq \sqrt[\phi]{\mathcal{U}_\ell} & \mathcal{S}.\mathsf{Tm}_0\ A \triangleq \phi \to [A] \\ \mathcal{S}.\# & \triangleq \phi & \mathcal{S}.\mathsf{Tm}_\omega\ A \triangleq [A] \end{array}$$

For erased functions we interpret $\mathcal{S}.\Pi_0\ A\ B \triangleq (a : (\phi \to A)) \to B\ a$ *and for runtime functions we interpret* $\mathcal{S}.\Pi_\omega\ A\ B \triangleq (a : A) \to B\ (\mathsf{p}\ a)$*, both using the function type in* $\sqrt[\phi]{\mathcal{U}_\ell}$*. Because* $B : (\phi \to A) \to \sqrt[\phi]{\mathcal{U}_\ell}$ *in both cases, we must use the weakening map* p *in the* ω *case to 'forget' the runtime data of* a*.*

The $\mathsf{Fam}(\mathbf{Set})$ model, explicitly (🕜) We expand $\mathcal{S}$ in first-order morphism form: each context Γ is interpreted as a set $[\![\Gamma]\!]^0_{\mathcal{S}} : \mathbf{Set}$ (erased phase) and a family $[\![\Gamma]\!]^1_{\mathcal{S}} : [\![\Gamma]\!]^0_{\mathcal{S}} \to \mathbf{Set}$ (runtime phase). A substitution $\Gamma \to \Delta$ is a function $\sigma_0 : [\![\Gamma]\!]^0_{\mathcal{S}} \to [\![\Delta]\!]^0_{\mathcal{S}}$ (erased) and a family of functions $\sigma_1 : \forall\gamma_0.\ [\![\Gamma]\!]^1_{\mathcal{S}}\ \gamma_0 \to [\![\Delta]\!]^1_{\mathcal{S}}\ (\sigma_0\ \gamma_0)$ (runtime), displayed over the erased function. An ℓ-type in context Γ is interpreted as a family of ℓ-sets $[\![\Gamma]\!]^0_{\mathcal{S}} \to (A_0 : \mathbf{Set}_\ell) \times (A_1 : A_0 \to \mathbf{Set}_\ell)$ indexed only over $[\![\Gamma]\!]^0_{\mathcal{S}}$. This is the result of expanding a map into the squashed universe. An erased term of type A in context Γ is interpreted as a section of the erased components $(\gamma_0 : [\![\Gamma]\!]^0_{\mathcal{S}}) \to [\![A]\!]^0_{\mathcal{S}}\ \gamma_0$, while a runtime term is interpreted as a full section $(\sigma_0 : (\gamma_0 : [\![\Gamma]\!]^0_{\mathcal{S}}) \to [\![A]\!]^0_{\mathcal{S}}\ \gamma_0) \times (\sigma_1 : \forall\gamma_0.\ [\![\Gamma]\!]^1_{\mathcal{S}}\ \gamma_0 \to [\![A]\!]^1_{\mathcal{S}}\ (\sigma_0\ \gamma_0))$. The sort $\# \in \Gamma$ is interpreted as $[\![\Gamma]\!]_{\mathcal{S}} \to \phi$ which is equivalent to $\forall\gamma_0.\ [\![\Gamma]\!]^1_{\mathcal{S}}\ \gamma_0 \to \mathbb{0}$; the assertion that the runtime part of the context is uninhabited. Finally, we can compute the context formers as:

$$\begin{array}{lll} [\![\bullet]\!]_{\mathcal{S}} & \triangleq \mathbb{1} & \simeq (\mathbb{1}, \lambda_.\ \mathbb{1}) \\ [\![\Gamma \rhd_0 A]\!]_{\mathcal{S}} & \triangleq (\gamma : [\![\Gamma]\!]_{\mathcal{S}}) \times (\phi \to [\![A]\!]_{\mathcal{S}}\ (\mathsf{p}\ \gamma)) & \simeq ((\gamma_0 : [\![\Gamma]\!]^0_{\mathcal{S}}) \times [\![A]\!]^0_{\mathcal{S}}\ \gamma_0,\ \lambda(\gamma_0, a_0).\ [\![\Gamma]\!]^1_{\mathcal{S}}\ \gamma_0) \\ [\![\Gamma \rhd_\omega A]\!]_{\mathcal{S}} & \triangleq (\gamma : [\![\Gamma]\!]_{\mathcal{S}}) \times [\![A]\!]_{\mathcal{S}}\ (\mathsf{p}\ \gamma) & \simeq ((\gamma_0 : [\![\Gamma]\!]^0_{\mathcal{S}}) \times [\![A]\!]^0_{\mathcal{S}}\ \gamma_0, \\ & & \quad \lambda(\gamma_0, a_0).\ (\gamma_1 : [\![\Gamma]\!]^1_{\mathcal{S}}\ \gamma_0) \times [\![A]\!]^1_{\mathcal{S}}\ \gamma_0\ a_0) \\ [\![\Gamma \rhd \#]\!]_{\mathcal{S}} & \triangleq [\![\Gamma]\!]_{\mathcal{S}} \times \phi & \simeq ([\![\Gamma]\!]^0_{\mathcal{S}}, \lambda_.\ \mathbb{0}) \end{array}$$

Erased context extension only extends the erased part of the context, runtime context extension extends both parts, and adding # makes the runtime part empty.

Models in Grothendieck toposes The construction of $\mathcal{S}$ can be generalised beyond $\mathsf{Fam}(\mathbf{Set})$, to an arbitrary Grothendieck topos $\mathbf{G}$ that supports squashed universes. Gratzer, Shulman and Sterling [18] have shown (classically) that any Grothendieck topos $\mathbf{G}$ admits a lifting of $\{\mathbf{Set}_\ell\}_{\ell<\omega}$ where each $\mathcal{U}_\ell$ contains all ℓ-small type families; Streicher [43] showed this constructively for presheaf toposes. So all Grothendieck toposes support universes. Which support squashed universes? We have written $\sqrt[\phi]{\mathcal{U}}$ to imply that $\sqrt[\phi]{-}$ is a functor. For $\mathsf{Fam}(\mathbf{Set})$, it takes an object $X \triangleq (X_0, X_1)$ to $((x : X_0) \times X_1\ x,\ \lambda_.\mathbb{1})$. It is uniquely characterised by the fact that it is the *right adjoint* to exponentiation by ϕ:

$$(\phi \to -) \dashv \sqrt[\phi]{-}$$

This construction has been studied in the context of type theory before [33, 36, 41, 37], frequently denoted by the square root $\sqrt{-}$ symbol. When a proposition ϕ has a right adjoint, it is called *tiny*. Therefore, if $\mathbf{G}$ has a tiny proposition ϕ, it supports squashed universes with respect to ϕ. This has been observed by Sterling [38] for essentially the same purpose. This is not an overly restrictive condition either: in a presheaf topos $\mathbf{Psh}(C)$, a representable $\mathsf{y}X$ is tiny whenever C has products with X. As a result, any Grothendieck topos with a tiny proposition supports a model of TT_0. We leave spelling out the details of the general construction for future work. This, along with the satisfaction of the realignment axiom [18], would justify the use of synthetic Tait computability [40] for the metatheory of TT_0.

7 Code extraction

The main purpose of TT_0 is to provide a practical language for programming with dependent types, so it is useful to be able to extract executable code from TT_0 programs. In particular, the code extraction process should erase all erased terms, and preserve the computational behaviour of runtime terms. In this section, we show how to extract code from TT_0 programs by interpreting into a *presheaf model* of TT_0 over the untyped lambda calculus.

Definition 21 (Untyped lambda calculus)**.** *The untyped lambda calculus λ quotiented by $\beta\eta$-equality is a SOGAT given by eq.* (1)*. Its initial GAT model is the CwF $\mathbf{0}_\lambda$, where contexts are natural numbers, substitutions $\mathbf{0}_\lambda.\mathsf{Sub}\ n\ m$ are m-tuples of lambda terms with n free variables, types are trivial (a single type $\star$), and terms $\mathbf{0}_\lambda.\mathsf{Tm}\ n$ are untyped lambda terms with n free variables [13]. This CwF supports Π types by lambda abstraction and application, and (positive) Σ types, natural numbers, and other data types by Church encoding.*

Definition 22 (Code extraction model of TT_0 🗲)**.** *The code extraction model $\mathcal{E}$ interprets TT_0 into the base category of presheaves over the syntax of the untyped lambda calculus, $\mathbf{Psh}(\mathbf{0}_\lambda)$, meaning it is a (Σ, Π_R)-CwF morphism $\mathcal{E} : \mathsf{TT}_0 \to \mathbf{Psh}_{\omega+1}(\mathbf{Psh}(\mathbf{0}_\lambda))$ where:*

$$\begin{aligned} \mathcal{E}.\mathsf{Ty}_\ell &\triangleq \mathbb{1} & \mathcal{E}.\mathsf{Tm}_0\ A &\triangleq \mathbb{1} \\ \mathcal{E}.\# &\triangleq \mathbb{0} & \mathcal{E}.\mathsf{Tm}_\omega\ A &\triangleq \mathsf{y}\mathbf{0}_\lambda.\mathsf{Tm} \end{aligned}$$

We interpret the runtime *(mode ω) Π and Σ as the corresponding Church-encoded untyped structures in $\mathbf{Psh}(\mathbf{0}_\lambda)$, while the* erased *ones disappear. For*

example, we have

$$\mathcal{E}.\Pi_0\ A\ B \triangleq \star \quad \textit{(all types are trivial)} \qquad \mathcal{E}.\mathsf{lam}_0\ t \triangleq t\ \star \qquad \mathcal{E}.\mathsf{app}_0\ f\ a \triangleq f$$

The erasure coercions $\uparrow$ *and* $\downarrow$ *reduce to the* ex falso quodlibet *principle and the terminal map respectively. Universes disappear as well, since they only exist in the erased fragment.*

The double presheaf codomain is needed because we map the representable sort $\#$ to $\mathbb{0}$ which is not representable in $\mathbf{Psh}(\mathbf{0}_\lambda)$ (there is no $\mathbf{0}_\lambda$-context n such that $\mathsf{y}n \simeq \mathbb{0}$).[1] Upon unfolding the GAT model $\widetilde{\mathcal{E}} : \mathsf{TT}_0^{\mathsf{fo}} \to \mathbf{Set}$ corresponding to the above, we can compute that contexts are $\mathbf{0}_\lambda$-presheaves, where context extensions are interpreted as:

$$\begin{aligned}
[\![\bullet]\!]_{\mathcal{E}}\ n &= \mathbb{1} & [\![\Gamma \rhd_0 A]\!]_{\mathcal{E}}\ n &= [\![\Gamma]\!]_{\mathcal{E}}\ n \times \mathbb{1}\ (\simeq [\![\Gamma]\!]_{\mathcal{E}}\ n)\\
[\![\Gamma \rhd_\omega A]\!]_{\mathcal{E}}\ n &= [\![\Gamma]\!]_{\mathcal{E}}\ n \times \mathbf{0}_\lambda.\mathsf{Tm}\ n & [\![\Gamma \rhd \#]\!]_{\mathcal{E}}\ n &= [\![\Gamma]\!]_{\mathcal{E}}\ n \times \mathbb{0}\ (\simeq \mathbb{0})
\end{aligned}$$

Up to isomorphism, adding an erased variable does nothing, adding a runtime variable adds a lambda term, and adding an erasure marker makes the context uninhabited.

To extract a program from a closed TT_0 term $t : \mathbf{0}_{\mathsf{TT}_0}.\mathsf{Tm}_\omega \bullet A$, we interpret it in the code extraction model to get a closed lambda term:

$$[\![t]\!]_{\mathcal{E}} : \mathcal{E}.\mathsf{Tm}_\omega\ [\![\bullet]\!]_{\mathcal{E}}\ [\![A]\!]_{\mathcal{E}} = \mathsf{Hom}_{\mathbf{Psh}(\mathbf{0}_\lambda)}(\mathbb{1}, \mathbf{0}_\lambda.\mathsf{Tm}) \simeq \mathsf{Hom}_{\mathbf{Psh}(\mathbf{0}_\lambda)}(\mathsf{y}0, \mathbf{0}_\lambda.\mathsf{Tm}) \simeq \mathbf{0}_\lambda.\mathsf{Tm}\ 0\,.$$

To interpret open terms, we observe that syntactic contexts which do not contain an erasure marker are representable in $\mathbf{Psh}(\mathbf{0}_\lambda)$.

Lemma 23. *If* $\# \notin \Gamma$ *(in* TT_0 *syntax), then* $[\![\Gamma]\!]_{\mathcal{E}}$ *is representable – there is a natural number* c_Γ *which counts the runtime bindings in* Γ*, satisfying*

$$[\![\Gamma]\!]_{\mathcal{E}} \simeq \mathsf{y}c_\Gamma\,.$$

Proof. By induction on contexts $\Gamma : \mathbf{0}_{\mathsf{TT}_0}.\mathsf{Con}$. □

Corollary 24. *For* $\# \notin \Gamma$*, there is an extraction map* $|-| : \mathbf{0}_{\mathsf{TT}_0}.\mathsf{Tm}_\omega\ \Gamma\ A \to \mathbf{0}_\lambda.\mathsf{Tm}\ c_\Gamma$.

Proof.

$$\begin{aligned}
[\![t]\!]_{\mathcal{E}} :\ \mathcal{E}.\mathsf{Tm}_\omega\ [\![\Gamma]\!]_{\mathcal{E}}\ [\![A]\!]_{\mathcal{E}} &= \mathsf{Hom}_{\mathbf{Psh}(\mathbf{0}_\lambda)}([\![\Gamma]\!]_{\mathcal{E}}, \mathbf{0}_\lambda.\mathsf{Tm})\\
&\simeq \mathsf{Hom}_{\mathbf{Psh}(\mathbf{0}_\lambda)}(\mathsf{y}c_\Gamma, \mathbf{0}_\lambda.\mathsf{Tm})\\
&\simeq \mathbf{0}_\lambda.\mathsf{Tm}\ c_\Gamma\,.
\end{aligned}$$

□

This is the 'purest' code extraction model we can formulate; in practice, we would choose a richer untyped target that includes primitives for pairs and inductive types (and one that would support full negative pairs), but its construction would be entirely analogous.

[1] Alternatively, we could form a *higher-order model*, followed by contextualisation [9] to get a GAT model in $\mathbf{Psh}(\mathbf{0}_\lambda)$.

7.1 Correctness of code extraction

To show that code extraction preserves the computational behaviour of TT_0 programs, we set up a logical relation between the code extraction model $\mathcal{E}$ (definition 21) and the logical interpretation in **Set**. We do so by building a model $\mathsf{Gl}(F)$ of TT_0 extended with natural numbers Nat in a glued category. This model is given by gluing along a morphism of TT_0-models $F : \mathbf{0}_{\mathsf{TT}_0} \to \mathcal{S}$. It is the result of combining two other morphisms. The *first* morphism is the composite

$$\mathbf{0}_{\mathsf{TT}_0} \xrightarrow{[\![-]\!]_{\mathcal{E}}} \mathcal{E} \xrightarrow{P \mapsto P\,0} \mathcal{S}_{\mathbf{Set}}$$

which evaluates a $\mathbf{0}_\lambda$-presheaf produced by code extraction at the empty context 0 (in other words, the global sections pseudo-morphism [27]). Its target is $\mathcal{S}_{\mathbf{Set}}$, the standard model of TT_0 valued in **Set** rather than $\mathsf{Fam}(\mathbf{Set})$ with # interpreted by the singleton set $\mathbb{1}$. The *second* morphism is the interpretation morphism $[\![-]\!]_{\mathcal{S}_{\mathbf{Set}}} : \mathbf{0}_{\mathsf{TT}_0} \to \mathcal{S}_{\mathbf{Set}}$ itself. These two define the morphism F, valued in the standard $\mathsf{Fam}(\mathbf{Set})$ model $\mathcal{S}$, via:

$$\mathbf{0}_{\mathsf{TT}_0} \xrightarrow{\Gamma \mapsto ([\![\Gamma]\!]_{\mathcal{S}_{\mathbf{Set}}}, \lambda_.\ [\![\Gamma]\!]_{\mathcal{E}}\, 0)} \mathcal{S}$$

sending a context Γ to the family $F\Gamma \triangleq ([\![\Gamma]\!]_{\mathcal{S}_{\mathbf{Set}}}, \lambda_.\ [\![\Gamma]\!]_{\mathcal{E}}\ 0)$. The standard **Set** interpretation is at the base of each object, and the code extraction is at the fibers. From F, we can construct a *displayed* TT_0 model $\mathsf{Gl}(F)$ analogously to the construction of Kaposi et al. [27].

The underlying category of $\mathsf{Gl}(F)$ is $\mathsf{Fam}(\mathbf{Set}) \downarrow F$ (where F really means the *underlying functor* between categories of contexts). Each context in $\mathsf{Gl}(F)$ consists of:

- a syntactic context $\Gamma : \mathbf{0}_{\mathsf{TT}_0}.\mathsf{Con}$
- a 'base' predicate $\Gamma_0^{\approx} : [\![\Gamma]\!]_{\mathcal{S}_{\mathbf{Set}}} \to \mathbf{Set}_\omega$
- a 'fiber' predicate $\Gamma_1^{\approx} : (\gamma_{\mathcal{S}} : [\![\Gamma]\!]_{\mathcal{S}_{\mathbf{Set}}}) \to \Gamma_0^{\approx}\ \gamma_{\mathcal{S}} \to [\![\Gamma]\!]_{\mathcal{E}}\ 0 \to \mathbf{Set}_\omega$

This comes with an evident projection morphism $(\Gamma, \Gamma_0^{\approx}, \Gamma_1^{\approx}) \mapsto \Gamma$ into the syntax $\mathbf{0}_{\mathsf{TT}_0}$, which by initiality has a section. This can be approximately thought of as a single binary relation that relates the set interpretation with code extraction. The nuance is that, in order to handle universes correctly, which are erased but must still carry logical predicate data, we need a base predicate $\Gamma_0^{\approx}$ which exists even in erased contexts to store them.

The sorts of the displayed model correspond to the induction motives of the logical relation, which are presented in fig. 2. This can be thought of as the logical relation version of the model described in section 6; the interpretation of types is 'squashed' because it is indexed by erased contexts only, but packs both base and fiber data. The main interesting component for our purposes is the type of natural numbers, where we relate the two interpretations in the fiber:

$$\mathsf{Nat}^{\approx} \triangleq (\lambda\gamma_0, n_{\mathsf{s}}.\ \mathbb{1},\ \lambda\{n_{\mathsf{s}}\}, n_{\mathsf{e}}, \star.\ n_{\mathsf{e}} = \mathsf{succ}^{n_{\mathsf{s}}}\ \mathsf{zero})$$

where $a^n\ b \triangleq \mathsf{rec}_{\mathbb{N}}\ b\ a\ n$. We omit the rest of the interpretation; see our Agda formalisation (𝒻).[2] From this model we obtain various useful correctness

[2]The formalisation works internally to $\mathbf{Psh}(\mathbf{0}_\lambda)$, which allows us to directly use a second-order model of λ rather than closed first-order terms. We include some notes there about its relation to this version.

$$
\begin{array}{ll}
(\Gamma : \mathsf{Con})^{\approx} & : (\Gamma_0^{\approx} : [\![\Gamma]\!]_{\mathcal{S}_{\mathbf{Set}}} \to \mathbf{Set}_\omega) \times (\Gamma_1^{\approx} : \forall \gamma_{\mathcal{S}}.\ [\![\Gamma]\!]_{\mathcal{E}}\ 0 \to \Gamma_0^{\approx}\ \gamma_{\mathcal{S}} \to \mathbf{Set}_\omega) \\
(\sigma : \mathsf{Sub}\ \Gamma\ \Delta)^{\approx} & : (\sigma_0^{\approx} : \forall \gamma_{\mathcal{S}}.\ \Gamma_0^{\approx}\ \gamma_{\mathcal{S}} \to \Delta_0^{\approx}\ ([\![\sigma]\!]_{\mathcal{S}_{\mathbf{Set}}}\ \gamma_{\mathcal{S}})) \\
& \quad \times (\sigma_1^{\approx} : \forall \gamma_{\mathcal{E}}, \gamma_0.\ \Gamma_1^{\approx}\ \gamma_{\mathcal{E}}\ \gamma_0 \to \Delta_1^{\approx}\ ([\![\sigma]\!]_{\mathcal{E}}\ 0\ \gamma_{\mathcal{E}})\ (\sigma_0^{\approx}\ \gamma_0)) \\
(A : \mathsf{Ty}_\ell\ \Gamma)^{\approx} & : (A_0^{\approx} : \forall \gamma_{\mathcal{S}}.\ \Gamma_0^{\approx}\ \gamma_{\mathcal{S}} \to [\![A]\!]_{\mathcal{S}_{\mathbf{Set}}}\ \gamma_{\mathcal{S}} \to \mathbf{Set}_\ell) \\
& \quad \times (A_1^{\approx} : \forall \gamma_0, a_{\mathcal{S}}.\ \mathbf{0}_{\lambda}.\mathsf{Tm}\ 0 \to A_0^{\approx}\ \gamma_0\ a_{\mathcal{S}} \to \mathbf{Set}_\ell) \\
(a : \mathsf{Tm}_0\ \Gamma\ A)^{\approx} & : \forall \gamma_{\mathcal{S}}.\ (\gamma_0 : \Gamma_0^{\approx}\ \gamma_{\mathcal{S}}) \to A_0^{\approx}\ \gamma_0\ ([\![a]\!]_{\mathcal{S}_{\mathbf{Set}}}\ \gamma_{\mathcal{S}}) \\
(a : \mathsf{Tm}_\omega\ \Gamma\ A)^{\approx} & : (a_0^{\approx} : \forall \gamma_{\mathcal{S}}.\ (\gamma_0 : \Gamma_0^{\approx}\ \gamma_{\mathcal{S}}) \to A_0^{\approx}\ \gamma_0\ ([\![a]\!]_{\mathcal{S}_{\mathbf{Set}}}\ \gamma_{\mathcal{S}})) \\
& \quad \times (a_1^{\approx} : \forall \gamma_{\mathcal{E}}, \gamma_0.\ \Gamma_1^{\approx}\ \gamma_{\mathcal{E}}\ \gamma_0 \to A_1^{\approx}\ ([\![a]\!]_{\mathcal{E}}\ 0\ \gamma_{\mathcal{E}})\ (a_0^{\approx}\ \gamma_0)) \\
(p : \# \in \Gamma)^{\approx} & : [\![\Gamma]\!]_{\mathcal{E}}\ 0 \to \mathbb{0}
\end{array}
$$

Figure 2: The motives of the logical relation for code extraction correctness.

properties of extraction. Below, we write $|-|$ for the code extraction of closed terms, $[\![-]\!]$ for the **Set** interpretation of closed terms, and operate purely in the syntax $\mathbf{0}_{\mathsf{TT}_0}$.

Theorem 25 (Canonicity 🦉)**.** *Every closed term* $n : \mathsf{Tm}_\omega \bullet \mathsf{Nat}$ *is extracted to the numeral of its set interpretation:* $|n| = \mathtt{succ}^{[\![n]\!]}\ \mathtt{zero}$.

Theorem 26 (Tracking 🦉)**.** *The extraction of any syntactic function tracks its semantic interpretation – for all* $f : \mathsf{Tm}_\omega \bullet (\Pi\ \mathsf{Nat}\ \mathsf{Nat})$ *and all* $k : \mathbb{N}$, $\mathtt{app}\ |f|\ (\mathtt{succ}^k\ \mathtt{zero}) = \mathtt{succ}^{[\![f]\!]\ k}\ \mathtt{zero}$.

Theorem 27 (Non-interference 🦉)**.** *Every erased function is constant at runtime – for all* $f : \mathsf{Tm}_\omega \bullet (\Pi_0\ \mathsf{Nat}\ \mathsf{Nat})$*, there exists a* $k : \mathbb{N}$ *such that for all* $x : \mathsf{Tm}_0 \bullet \mathsf{Nat}$, $|\mathsf{app}_0\ f\ x| = |f| = \mathtt{succ}^k\ \mathtt{zero}$*. Moreover,* $[\![f]\!]$ *is the constant function returning* k*.*

8 Implementation

We have implemented a toy elaborator for type theory with erasure. This is based on András Kovács' `elaboration-zoo` which contains toy implementations of dependent type theory. Our implementation is a modification of the implementation of implicit arguments and metavariables [29]. The surface language is essentially the language presented in section 2, with mode-aware Π types and a single universe $\mathsf{U} : \mathsf{U}$. The coercions $\uparrow/\downarrow$ and the marker $\#$ are inserted automatically; the user never interacts with them.

In the repository, we include two variants of the elaboration algorithm: one which *inserts* coercions during elaboration, and one which keeps coercions implicit. The latter is closer to existing implementations of erasure. We keep the former as a proof of concept that it is possible to have a structural phase distinction which can be elaborated from a 'substructural' source language. In both cases, we make a simplification to the representation of contexts: the theory TT_0 as presented features context extensions by $\#$ which can end up anywhere

in the context, and can appear multiple times. However, # is a proposition, so it doesn't matter which particular witness we use. It is therefore sufficient to keep a boolean flag indicating the mere *presence* of # in a context otherwise containing only $0/\omega$ bindings. This simplifies the handling of variables, since we don't need to offset de Bruijn indices/levels by #.

The predominant source of complexity in elaborating such languages is the pattern unification algorithm that solves metavariables. Although there are theoretical foundations of pattern unification for type theory [2], such a formal analysis has not been performed for languages with erasure or other modalities. The implementations of pattern unification in the wild are thus 'engineering efforts', which can go wrong. Despite checking quantities only *after* the whole program has been elaborated, Idris 2 sometimes solves metas in a weaker quantity than required, which can lead to undefined behaviour at runtime [16]. On the other hand, Agda sometimes fails to detect unsolvable metas in the presence of erasure [14].

Luckily, because our implementation is based on a structural core, we can directly reuse the theory of pattern unification to obtain a correct implementation. Our unification algorithm does not require a separate mode check after typechecking as opposed to Idris. It also does not need a dedicated generalisation mechanism for promoting erased metavariables to runtime metavariables as opposed to Agda. In the repository, we include some test cases that otherwise fail in these languages due to the incompleteness of their respective mechanisms (they are the test cases in the issues linked above). We have justified our formulation of pattern unification by some semi-formal notes in the same repository. The core idea is that the process of renaming and performing occurrence checking on candidate solutions handles not only regular variables, but also witnesses of the erasure marker # (which are morally also just variables). As a result, we only need one kind of metavariable (runtime) and generalisation is no longer necessary.

9 Related work

Our approach to erasure is based on *synthetic phase distinctions*, pioneered by Sterling and Harper [42], whose roots go back to the work on phase distinctions of Cardelli [11] and Harper, Mitchell, and Moggi [21]. Cardelli's work is the closest to erasure in terms of purpose, but is formulated as an 'indexed' type theory. In his thesis [40], Sterling develops a theory of synthetic phase distinctions as a tool for constructing logical relations (synthetic Tait computability), but with various applications to programming languages. Most recently, Grodin et al. [19] have showcased the possible use cases of a language with phase distinctions. In such settings one has access to open as well as closed modalities, corresponding to open and closed subtoposes of the topos in which the language has semantics. Using this terminology, our formulation of erasure is the open modality for the proposition #. The closest work along these lines to ours is in the form of a blog post by Sterling [38], which contains some ideas about how synthetic Tait computability relates to QTT; in particular, he observes the need for squashed universes.

Erasure in dependent types was explored by Mishra-Linger et al. [35] in the context of *pure type systems*. With the work of Gundry and McBride [20] as

precursor, the modern approach to erasure has been QTT by McBride [34] and Atkey [7]. We intend to characterise the relationship between our theory and QTT, whose models are *quantitative* CwFs (QCwFs), in the future. For now, we make the observation that a *structural* QCwF (meaning with the semiring $\mathcal{R} = \{0 < \omega\}$) is an indexed CwF $U : \mathcal{L} \to \mathcal{C}$. Gluing along the reindexing functor $U^\star : \mathbf{Psh}(\mathcal{C}) \to \mathbf{Psh}(\mathcal{L})$ yields a presheaf category with a tiny proposition that models TT_0. In the other direction, given a TT_0-model $\mathcal{M}$, strictifying the pseudo-morphism $\mathcal{M} \to \mathcal{M}/\#$ into the slice CwF $\mathcal{M}/\#$ yields a structural QCwF.

More recently, Danielsson has explored some constructions in type theory with erasure [15], and Abel et al. have explored its integration with cubical type theory for Cubical Agda [1]. We expect that our system can extend the SOGAT of cubical type theory [45, 4.6.3] with a mode split for terms, and thus replicate Abel's system structurally. Favier [17] has shown that erasure behaves like an open modality in Agda, showing a synthetic Artin fracture theorem. Besides this, there has also been work on theories with 'mode splits', notably type theory with colours [26]. This style of system, where terms at each mode need not be the same, can be replicated in our system; we are free to add equations that apply only under the # marker, collapsing data in the erased phase that otherwise exists at the runtime phase.

10 Conclusion

We have developed a fully structural theory of erasure using the formalism of SOGATs, and explored various syntactic and semantic models. In the future, we would like to explore more extensions to the theory. The most immediate is to support runtime types. This is relatively straightforward to add and simplifies the semantics by avoiding the need for squashed universes; we are mostly interested in exploring the utility of this feature for programming. Another extension is to add more phase distinctions. Suppose we allow equations like β-reduction only under #. We could then add a second *disjoint* phase distinction \$ to internalise the *code extraction* morphism. Doing so would allow us to control and reason about compilation output internally to the language, for example to specify runtime optimisations.

Complete fixed author formal proof continuation (MIT)

Fixed author commit b840465809e71c5fc469fcd8b267a8191e931341. The original source files follow without executing Agda. The author models the target lambda calculus internally in a presheaf category; the original target-interface postulates and comments remain visible.

`Utils.agda`

```
module Utils where

open import Axiom.Extensionality.Propositional using (Extensionality)
open import Level using (Level; _⊔_; suc)
open import Data.Product
```

```
open import Agda.Builtin.Sigma
open import Data.Empty renaming (⊥ to Empty)

-- Acknowledgements to Szumi Xie for many proof irrelevant equality utilities:
-- https://github.com/szumixie/single-subst/blob/main/TT/Lib.agda

private variable
  ℓ ℓ' : Level
  A A' : Set ℓ
  P P' : Prop
  B B' : A → Set ℓ
  C : P → Set ℓ
  x y z : A

-- Prop primitives

data ⊤ : Prop where
    tt : ⊤

data ⊥ : Prop where

exfalso : ⊥ → A
exfalso ()

exfalso-prop : ∀ {P : Prop ℓ} → ⊥ → P
exfalso-prop ()

exfalso-empty-prop : Empty → P
exfalso-empty-prop ()

infix 4 _≡_
data _≡_ {A : Set ℓ} (x : A) : A → Prop ℓ where
  instance refl : x ≡ x

infix 4 _=_
data _=_ {P : Prop ℓ} (x : P) : P → Prop ℓ where
  instance reflP : x = x

{-# BUILTIN REWRITE _≡_ #-}

private variable
  p q : A ≡ A'

record _≃_ (A : Set ℓ) (B : Set ℓ) : Set ℓ where
  field
    to : A → B
    from : B → A
    to-from : ∀ x → to (from x) ≡ x
    from-to : ∀ x → from (to x) ≡ x

open _≃_ public

record _true (A : Prop ℓ) : Set ℓ where
  constructor by
  field
    witness : A

open _true public

record ΣProp {ℓ ℓ'} (A : Prop ℓ) (B : A → Prop ℓ') : Prop (ℓ ⊔ ℓ') where
  constructor _,P_
  field
    fst : A
    snd : B fst

open ΣProp public

-- Some fragment of OTT

postulate
  coe₀ : A ≡ A' → A → A'
  funext : ∀ {f g : (x : A) → B x} → (∀ x → f x ≡ g x) → f ≡ g
  propfunext : ∀ {f g : (x : P) → C x} → (∀ x → f x ≡ g x) → f ≡ g

opaque
  coe : A ≡ A' → A → A'
```

```
  coe = coe₀


cong : (f : A → A') → x ≡ y → f x ≡ f y
cong f refl = refl

cong₂ : ∀ {B C : Set ℓ} {z w : B} → (f : A → B → C) → x ≡ y → z ≡ w → f x z ≡ f y w
cong₂ f refl refl = refl

sym : x ≡ y → y ≡ x
sym refl = refl

trans : x ≡ y → y ≡ z → x ≡ z
trans refl p = p

private
  Π : (A : Set ℓ) → (B : A → Set ℓ') → Set (ℓ ⊔ ℓ')
  Π A B = (a : A) → B a

-- We don't add computation rules for the equality type---since it is inductively
-- defined that would break things.
postulate
  coe-Σ : (Σ A B ≡ Σ A' B') → (ΣProp (A ≡ A') (λ p → ∀ x → B x ≡ B' (coe p x)))
  coe-pair :
    {a : A} {b : B a}
    {p : Σ A B ≡ Σ A' B'}
    → coe₀ p (a , b) ≡ (coe (coe-Σ p .fst) a , coe (coe-Σ p .snd a) b)
  {-# REWRITE coe-pair #-}

  coe-Π : (((a : A) → B a) ≡ ((a : A') → B' a))
    → (ΣProp (A' ≡ A) (λ p → ∀ x → B (coe p x) ≡ B' x))
  coe-lam :
    {f : Π {ℓ} {ℓ'} A B}
    {p : ((a : A) → B a) ≡ ((a : A') → B' a)}
    → coe₀ p f ≡ λ a → coe (coe-Π p .snd _) (f (coe (coe-Π p .fst) a))
  {-# REWRITE coe-lam #-}

-- Adding this as the last rule because it's the most general. Otherwise
-- the typechecker dies
postulate
  coe₀-eq : coe₀ refl x ≡ x
  {-# REWRITE coe₀-eq #-}

-- Equality helpers

subst : (P : A → Set ℓ) (p : x ≡ y) → P x → P y
subst P p a = coe (cong P p) a

infix 4 _≡[_]_

_≡[_]_ : ∀ (a : A) (p : A ≡ A') (b : A') → Prop _
_≡[_]_ a p b = coe p a ≡ b

opaque
  unfolding coe

  Σ≡ : {p₁ p₂ : Σ A B} (p : p₁ .proj₁ ≡ p₂ .proj₁) → subst B p (p₁ .proj₂) ≡ (p₂ .proj₂) → p₁ ≡ p₂
  Σ≡ refl refl = refl

  ≡Σ : {p₁ p₂ : Σ A B} (p : p₁ ≡ p₂)
    → ΣProp (p₁ .proj₁ ≡ p₂ .proj₁) (λ p → subst B p (p₁ .proj₂) ≡ (p₂ .proj₂))
  ≡Σ refl = refl ,P refl

  reflᴰ : x ≡[ refl ] x
  reflᴰ = refl

  symᴰ : x ≡[ p ] y → y ≡[ sym p ] x
  symᴰ {p = refl} refl = refl

  transᴰ : x ≡[ p ] y → y ≡[ q ] z → x ≡[ trans p q ] z
  transᴰ {p = refl} {q = refl} refl m = m

  congᴰ : (B : A → Set ℓ) → (f : (a : A) → B a) → (p : x ≡ y) → f x ≡[ cong B p ] f y
  congᴰ _ _ refl = refl

  dep : x ≡ y → x ≡[ refl ] y
```

```
  dep p = p

  undep : x ≡[ refl ] y → x ≡ y
  undep p = p

  movel : x ≡ coe (sym q) y → x ≡[ q ] y
  movel {q = refl} refl = refl

  splitl : x ≡[ trans p q ] y → coe p x ≡[ q ] y
  splitl {p = refl} q = q

  splitr : x ≡[ trans p (sym q) ] y → x ≡[ p ] coe q y
  splitr {p = refl} {q = refl} refl = refl

  merger : x ≡[ p ] coe q y → x ≡[ trans p (sym q) ] y
  merger {p = refl} {q = refl} refl = refl


opaque
  unfolding coe

  ap-→ : ∀ {C C' : Set ℓ} → A ≡ A' → C ≡ C' → (A → C) ≡ (A' → C')
  ap-→ refl refl = refl

  ap-$ : ∀ {f g : (x : A) → B x} → f ≡ g → ∀ x → f x ≡ g x
  ap-$ refl x = refl
```

Mode.agda

```
{-# OPTIONS --type-in-type #-}
module Mode where

open import Agda.Primitive
open import Utils

data Mode : Set where
  z : Mode
  ω : Mode

opaque
  _*_ : Mode → Mode → Mode
  j * ω = j
  ω * j = j
  z * j = z

variable
  i j : Mode

opaque
  unfolding _*_

  j*ω : j * ω ≡ j
  j*ω {z} = refl
  j*ω {ω} = refl

  ω*j : ω * j ≡ j
  ω*j {z} = refl
  ω*j {ω} = refl

  z*j : z * j ≡ z
  z*j {z} = refl
  z*j {ω} = refl

  j*z : j * z ≡ z
  j*z {z} = refl
  j*z {ω} = refl

-- Better definitional computation for _*_
{-# REWRITE j*ω ω*j z*j j*z #-}
```

Theories/CwFwE.agda

```
{-# OPTIONS --type-in-type --lossy-unification #-}
module Theories.CwFwE where

open import Agda.Primitive
open import Utils
open import Data.Unit renaming (⊤ to 𝟙)
open import Mode

-- Model

record CwFwE-sorts : Set where
  field
    Con : Set
    Sub : Con → Con → Set
    Ty : Con → Set
    #∈ : Con → Set
    Tm : ∀ Γ → Mode → Ty Γ → Set
    #-prop : ∀ {Γ} (p q : #∈ Γ) → p ≡ q

module in-CwFwE-sorts (s : CwFwE-sorts) where
  open CwFwE-sorts s
  variable
    Γ Γ' Δ Δ' θ θ' : Con
    σ σ' τ τ' ρ : Sub Γ Δ
    A A' B B' C : Ty Γ
    t t' u u' v : Tm Γ i A
    π π' ξ : #∈ Γ

  opaque
    unfolding coe

    ap-Subᶜ : Γ ≡ Γ' → Δ ≡ Δ' → Sub Γ Δ ≡ Sub Γ' Δ'
    ap-Subᶜ refl refl = refl

    ap-Tyᶜ : Γ ≡ Γ' → Ty Γ ≡ Ty Γ'
    ap-Tyᶜ refl = refl

    ap-#∈ᶜ : Γ ≡ Γ' → #∈ Γ ≡ #∈ Γ'
    ap-#∈ᶜ refl = refl

    ap-Tm : A ≡ A' → Tm Γ i A ≡ Tm Γ i A'
    ap-Tm refl = refl

    ap-Tmᶜ : (e : Γ ≡ Γ') → A ≡[ ap-Tyᶜ e ] A' → Tm Γ i A ≡ Tm Γ' i A'
    ap-Tmᶜ refl refl = refl

    ap-Tmᶜ' : (e : Γ ≡ Γ') → i ≡ j →  A ≡[ ap-Tyᶜ e ] A' → Tm Γ i A ≡ Tm Γ' j A'
    ap-Tmᶜ' refl refl refl = refl


  module core-utils
    (_[_]T : ∀ {Γ Δ} → (A : Ty Δ) → (σ : Sub Γ Δ) → Ty Γ)
    (_[_] : ∀ {Γ Δ i A} → (a : Tm Δ i A) → (σ : Sub Γ Δ) → Tm Γ i (A [ σ ]T))
    where
    opaque
      unfolding coe

      ap-[]T₀-impl : σ ≡ τ → A [ σ ]T ≡ A [ τ ]T
      ap-[]T₀-impl refl = refl

      ap-[]T₁-impl : A ≡ B → A [ σ ]T ≡ B [ σ ]T
      ap-[]T₁-impl refl = refl

  record CwFwE-core : Set where
    field
      id : Sub Γ Γ
      _∘_ : (σ : Sub Δ θ) → (τ : Sub Γ Δ) → Sub Γ θ
      assoc : ρ ∘ (σ ∘ τ) ≡ (ρ ∘ σ) ∘ τ
      id∘ : id ∘ σ ≡ σ
      ∘id : σ ∘ id ≡ σ

      • : Con
      ε : Sub Γ •
      ∃!ε : ε {Γ} ≡ σ
```

```
  -- All the equations on the #Є_ sort are technically unnecessary
  -- because it is a proposition, but we include them here anyway.

  _[_]T : (A : Ty Δ) → (σ : Sub Γ Δ) → Ty Γ
  _[_] : (t : Tm Δ i A) → (σ : Sub Γ Δ) → Tm Γ i (A [ σ ]T)
  _[_]# : (t : #Є Δ) → (σ : Sub Γ Δ) → #Є Γ
  [id]T : A [ id ]T ≡ A
  [id] : (t [ id ]) ≡[ ap-Tm {i = i} [id]T ] t
  [id]# : π [ id ]# ≡ π
  [∘]T : A [ σ ∘ τ ]T ≡ (A [ σ ]T) [ τ ]T
  [∘] : t [ σ ∘ τ ] ≡[ ap-Tm {i = i} [∘]T ] ((t [ σ ]) [ τ ])
  [∘]# : π [ σ ∘ τ ]# ≡ (π [ σ ]#) [ τ ]#

  -- Context extension for terms
  _▷[_]_ : (Γ : Con) → Mode → (A : Ty Γ) → Con

ap-[]T₀ : σ ≡ τ → A [ σ ]T ≡ A [ τ ]T
ap-[]T₀ = core-utils.ap-[]T₀-impl _[_]T _[_]

ap-[]T₁ : A ≡ B → A [ σ ]T ≡ B [ σ ]T
ap-[]T₁ = core-utils.ap-[]T₁-impl _[_]T _[_]

field
  p : Sub (Γ ▷[ i ] A) Γ
  q : Tm (Γ ▷[ i ] A) i (A [ p ]T)
  _,,_ : (σ : Sub Γ Δ) → (t : Tm Γ i (A [ σ ]T)) → Sub Γ (Δ ▷[ i ] A)
  ,∘ : (σ ,, t) ∘ ρ ≡ (σ ∘ ρ) ,, coe (ap-Tm {i = i} (sym [∘]T)) (t [ ρ ])
  p,q : p {Γ} {i} {A} ,, q ≡ id
  p∘, : p {Γ} {i} {A} ∘ (σ ,, t) ≡ σ
  q[,] : q [ σ ,, t ] ≡[ ap-Tm {i = i} (trans (sym [∘]T) (ap-[]T₀ p∘,)) ] t

  -- Context extension for #
  _▷# : (Γ : Con) → Con
  p# : Sub (Γ ▷#) Γ
  q# : #Є (Γ ▷#)
  _,#_ : (σ : Sub Γ Δ) → (π : #Є Γ) → Sub Γ (Δ ▷#)
  ,#∘ : (σ ,# π) ∘ ρ ≡ (σ ∘ ρ) ,# (π [ ρ ]#)
  p,#q : p# {Γ} ,# q# ≡ id
  p∘,# : p# ∘ (σ ,# π) ≡ σ
  q[,#] : q# [ σ ,# π ]# ≡ π

⟨_⟩ : (t : Tm Γ i A) → Sub Γ (Γ ▷[ i ] A)
⟨ t ⟩ = id ,, (t [ id ])

_⁺ : (σ : Sub Γ Δ) → Sub (Γ ▷[ i ] (A [ σ ]T)) (Δ ▷[ i ] A)
σ ⁺ = (σ ∘ p) ,, coe (ap-Tm (sym [∘]T)) q

_⁺# : (σ : Sub Γ Δ) → Sub (Γ ▷#) (Δ ▷#)
σ ⁺# = (σ ∘ p#) ,# q#

field
  -- Conversion between terms
  ↓ : Tm (Γ ▷#) ω (A [ p# ]T) → Tm Γ z A
  ↑ : Tm Γ z A → Tm (Γ ▷#) ω (A [ p# ]T)

  ↓[] : (↓ t) [ σ ] ≡ ↓
    (coe (ap-Tm (trans (sym [∘]T) (trans (ap-[]T₀ p∘,#) [∘]T)))
    (t [ σ ⁺# ]))
  -- The other direction is derivable (and I will not derive it)

  ↑↓ : ↑ (↓ t) ≡ t
  ↓↑ : ↓ (↑ t) ≡ t

-- Various congruence rules
opaque
  unfolding coe

  ap-▷[] : (p : Γ ≡ Δ) → A ≡[ ap-Tyᶜ p ] B → (Γ ▷[ i ] A) ≡ (Δ ▷[ i ] B)
  ap-▷[] refl refl = refl

  ap-id : (p : Γ ≡ Γ') → id {Γ} ≡[ ap-Subᶜ p p ] id {Γ'}
  ap-id refl = refl

  ap-ε : (p : Γ ≡ Γ') → ε {Γ} ≡[ ap-Subᶜ p refl ] ε {Γ'}
  ap-ε refl = refl
```

```
  ap-∘ : ∀ (p : Γ ≡ Γ') (q : Δ ≡ Δ') (r : θ ≡ θ') {σ τ}
    → σ ≡[ ap-Subᶜ p q ] σ'
    → τ ≡[ ap-Subᶜ r p ] τ'
    → σ ∘ τ ≡[ ap-Subᶜ r q ] σ' ∘ τ'
  ap-∘ refl refl refl refl refl = refl

  ap-[]T : ∀ (q : Δ ≡ Δ') (p : Γ ≡ Γ') → A ≡[ ap-Tyᶜ p ] A' → σ ≡[ ap-Subᶜ q p ] σ' → (A [ σ ]T) ≡[
  ↪  ap-Tyᶜ q ] (A' [ σ' ]T)
  ap-[]T refl refl refl refl = refl

  ap-p : (prΓ : Γ ≡ Γ')
    → (prA : A ≡[ ap-Tyᶜ prΓ ] A')
    → p {Γ} {i} {A} ≡[ ap-Subᶜ (ap-▷[] prΓ prA) prΓ ] p {Γ'} {i} {A'}
  ap-p refl refl = refl

  ap-q : ∀ (prΓ : Γ ≡ Γ')
    → (prA : A ≡[ ap-Tyᶜ prΓ ] A')
    → q {Γ} {i} {A} ≡[ ap-Tmᶜ (ap-▷[] prΓ prA) (ap-[]T (ap-▷[] prΓ prA) prΓ prA (ap-p prΓ prA) ) ] q
    ↪  {Γ'} {i} {A'}
  ap-q refl refl = refl

  ap-[] : ∀ (q : Δ ≡ Δ') (p : Γ ≡ Γ')
    → (prA : A ≡[ ap-Tyᶜ p ] A')
    → (prσ : σ ≡[ ap-Subᶜ q p ] σ')
    → t ≡[ ap-Tmᶜ p prA ] t'
    → (t [ σ ]) ≡[ ap-Tmᶜ q (ap-[]T q p prA prσ) ] (t' [ σ' ])
  ap-[] refl refl refl refl refl = refl

  ap-,, : ∀ (p : Γ ≡ Γ') (q : Δ ≡ Δ')
    {t : Tm Γ i (A [ σ ]T)}
    {t' : Tm Γ' i (A' [ σ' ]T) }
    → (prσ : σ ≡[ ap-Subᶜ p q ] σ')
    → (prA : A ≡[ ap-Tyᶜ q ] A')
    → t ≡[ ap-Tmᶜ p (ap-[]T p q prA prσ) ] t'
    → (σ ,, t) ≡[ ap-Subᶜ p (ap-▷[] q prA ) ] (σ' ,, t')
  ap-,, refl refl refl refl refl = refl

  ap-,# : ∀ (p : Γ ≡ Γ') (q : Δ ≡ Δ')
    → (prσ : σ ≡[ ap-Subᶜ p q ] σ')
    → π ≡[ ap-#∈ᶜ p ] π'
    → (σ ,# π) ≡[ ap-Subᶜ p (cong _▷# q) ] (σ' ,# π')
  ap-,# refl refl refl refl = refl

opaque
  unfolding coe
  ap-[]₀ : (p : σ ≡ τ) → t [ σ ] ≡[ ap-Tm (ap-[]T₀ p) ] t [ τ ]
  ap-[]₀ refl = refl

  ap-[]₁ : ∀ {t : Tm Γ i A} {u : Tm Γ i B} (p : A ≡ B) → t ≡[ ap-Tm p ] u → t [ σ ] ≡[ ap-Tm (ap-[]T₁ p)
  ↪  ] u [ σ ]
  ap-[]₁ refl refl = refl

p#∘p#,#q# : ((p# {Γ} ∘ p#) ,# q#) ≡ p#
p#∘p#,#q# = trans (trans (trans (undep (ap-,# refl refl reflᴰ (#-prop _ _))) (sym ,#∘))
 (cong (_∘ p#) p,#q)) (id∘)


↑[p#] : (↑ t) [ p# ] ≡ ↑ (t [ p# ])
↑[p#] = sym (trans (cong ↑ (undep (ap-[]₁ refl (dep (sym ↓↑)))))
  (trans (cong ↑ ↓[]) (trans ↑↓ (ap-[]₀ p#∘p#,#q#))))

↓[p#] : (↓ t) [ p# ] ≡ ↓ (t [ p# ])
↓[p#] = trans ↓[] ( cong ↓ (ap-[]₀ p#∘p#,#q#))

opaque
  ↓* : Tm Γ i A → Tm Γ z A
  ↓* {i = z} t = t
  ↓* {i = ω} t = ↓ (t [ p# ])

  ↓*[] : {t : Tm Γ i A} → (↓* t) [ σ ] ≡ ↓* (t [ σ ])
  ↓*[] {i = z} = refl
  ↓*[] {i = ω} = trans ↓[] (cong ↓ (transᴰ (symᴰ [∘]) (transᴰ (ap-[]₀ p∘,#) [∘])))

  ↓*↑ : {t : Tm Γ z A} → ↓* (↑ t) ≡ t [ p# ]
  ↓*↑ = trans (cong ↓ ↑[p#]) ↓↑
```

```
  opaque
    unfolding ↓* coe
    ap-↓* : (p : A ≡ B) → t ≡[ ap-Tm p ] u → ↓* t ≡[ ap-Tm p ] ↓* u
    ap-↓* refl refl = refl

  opaque
    unfolding ↓*

    ↓*q[,] : (↓* {i = i} q) [ σ ,, t ] ≡[ ap-Tm (trans (sym [∘]T) (ap-[]T₀ p∘,)) ] ↓* t
    ↓*q[,] = transᴰ (dep ↓*[]) (ap-↓* (trans (sym [∘]T) (ap-[]T₀ p∘,)) q[,])

    pz : Sub (Γ ▷[ i ] A) (Γ ▷[ z ] A)
    pz = p ,, ↓* q

    p,↓*q : (p {Γ} {_} {A} ,, ↓* {i = z} q) ≡ id
    p,↓*q = p,q

    -- These lemmas are needed for lam[]
    -- @@TODO: Remove these, use normal Π instead..
    pz∘⁺≡⁺∘pz : (_⁺ {Γ} {A = A} σ) ∘ pz {Γ} {ω} ≡ pz ∘ (σ ⁺)
    pz∘⁺≡⁺∘pz {σ = σ} = trans ,∘ (trans
      (undep (ap-,, refl refl (dep (trans (trans (sym assoc) (cong (σ ∘_) p∘,)) (sym p∘,)))
      reflᴰ (splitl (transᴰ {q = ap-Tm (ap-[]T₀ (sym p∘,))} (transᴰ (ap-[] refl refl (dep [∘]T) reflᴰ
      ↪  (splitl reflᴰ))
      (transᴰ q[,] (ap-↓* ( sym [∘]T) (splitr reflᴰ)))) (splitr (symᴰ ↓*q[,])))))))
      (sym ,∘))

    [pz][⁺]≡[⁺][pz] : (A [ σ ⁺ ]T) [ pz {Γ} {i} ]T ≡ (A [ pz ]T) [ σ ⁺ ]T
    [pz][⁺]≡[⁺][pz] {A = A} {Γ = Γ} {σ = σ} {i = z} =
      trans (ap-[]T₀ p,q)
      (trans [id]T ((ap-[]T₁ (sym (trans (ap-[]T₀ p,q) [id]T)))))
    [pz][⁺]≡[⁺][pz] {A = A} {Γ = Γ} {σ = σ} {i = ω} =
      trans (sym [∘]T) (trans (ap-[]T₀ pz∘⁺≡⁺∘pz) [∘]T)

module in-CwFwE-core (c : CwFwE-core) where
  open CwFwE-core c

  record Π-structure  : Set where
    field
    -- @@TODO: use normal Π instead (i.e. replace z with i). It is equivalent.. Will probably make some
    ↪  proofs easier too.
      Π : (i : Mode) → (A : Ty Γ) → (B : Ty (Γ ▷[ z ] A)) → Ty Γ
      Π[] : (Π i A B) [ σ ]T ≡ Π i (A [ σ ]T) (B [ σ ⁺ ]T)

      lam : (f : Tm (Γ ▷[ i ] A) ω (B [ pz ]T)) → Tm Γ ω (Π i A B)
      lam[] : (lam {i = i} t) [ σ ]
        ≡[ ap-Tm Π[] ] lam (coe (ap-Tm (sym [pz][⁺]≡[⁺][pz])) (t [ σ ⁺ ]))

      ap : (f : Tm Γ ω (Π i A B)) → Tm (Γ ▷[ i ] A) ω (B [ pz ]T)

      Πβ : ap {i = i} (lam t) ≡ t
      Πη : lam {i = i} (ap t) ≡ t

  record U-structure : Set where
    field
      U : Ty Γ
      U[] : U [ σ ]T ≡ U

      El : (t : Tm Γ z U) → Ty Γ
      El[] : (El t) [ σ ]T ≡ El (coe (ap-Tm U[]) (t [ σ ]))

      code : (A : Ty Γ) → Tm Γ z U
      code[] : (code A) [ σ ] ≡[ ap-Tm U[] ] code (A [ σ ]T)

      El-code : El (code A) ≡ A
      code-El : code (El t) ≡ t

    opaque
      unfolding coe
      ap-U : (p : Γ ≡ Γ') → U ≡[ ap-Tyᶜ p ] U
      ap-U refl = refl

      ap-El : (p : Γ ≡ Γ') → t ≡[ ap-Tmᶜ p (ap-U p) ] u → El t ≡[ ap-Tyᶜ p ] El u
      ap-El refl refl = refl
```

```
        ap-code : (p : Γ ≡ Γ') → A ≡[ ap-Tyᶜ p ] B → code A ≡[ ap-Tmᶜ p (ap-U p) ] code B
        ap-code refl refl = refl


record CwFwE : Set where
  field
    sorts : CwFwE-sorts
  open in-CwFwE-sorts sorts
  field
    core : CwFwE-core
  open in-CwFwE-core core
  field
    Π-str : Π-structure
    U-str : U-structure

-- Displayed model


record CwFwEᴰ-sorts (s : CwFwE-sorts) : Set where
  open CwFwE-sorts s
  field
    Conᴰ : Con → Set
    Subᴰ : ∀ {Γ Δ} → Conᴰ Γ → Conᴰ Δ → Sub Γ Δ → Set
    Tyᴰ : ∀ {Γ} → Conᴰ Γ → Ty Γ → Set
    #Єᴰ : ∀ {Γ} → Conᴰ Γ → #Є Γ → Set
    Tmᴰ : ∀ {Γ A} → (Γᴰ : Conᴰ Γ) → (i : Mode) → Tyᴰ Γᴰ A → Tm Γ i A → Set
    #-propᴰ : ∀ {Γ Γᴰ π π'} → (p : #Єᴰ {Γ} Γᴰ π) (q : #Єᴰ Γᴰ π') → (prf : π ≡ π') → p ≡[  cong (#Єᴰ _) prf  ]
    ↪  q

module in-CwFwEᴰ-sorts {s : CwFwE-sorts} (sᴰ : CwFwEᴰ-sorts s) (c : in-CwFwE-sorts.CwFwE-core s) where
  open CwFwE-sorts s
  open CwFwEᴰ-sorts sᴰ
  open in-CwFwE-sorts s
  open CwFwE-core c
  open in-CwFwE-core c
  variable
    Γᴰ Γᴰ' Δᴰ Δᴰ' θᴰ : Conᴰ Γ
    σᴰ σᴰ' τᴰ τᴰ' ρᴰ : Subᴰ Γᴰ Δᴰ σ
    Aᴰ Aᴰ' Bᴰ Cᴰ : Tyᴰ Γᴰ A
    tᴰ tᴰ' uᴰ uᴰ' vᴰ : Tmᴰ Γᴰ i Aᴰ t
    πᴰ : #Єᴰ Γᴰ π

  opaque
    unfolding coe

    ap-Conᴰᶜ : Γ ≡ Δ → Conᴰ Γ ≡ Conᴰ Δ
    ap-Conᴰᶜ refl = refl

    ap-Subᴰ : σ ≡ τ → Subᴰ Γᴰ Δᴰ σ ≡ Subᴰ Γᴰ Δᴰ τ
    ap-Subᴰ refl = refl

    ap-Subᴰᶜ : (pΓ : Γ ≡ Γ') → (pΔ : Δ ≡ Δ')
      → Γᴰ ≡[ ap-Conᴰᶜ pΓ ] Γᴰ'
      → Δᴰ ≡[ ap-Conᴰᶜ pΔ ] Δᴰ'
      → σ ≡[ ap-Subᶜ pΓ pΔ ] τ
      → Subᴰ Γᴰ Δᴰ σ ≡ Subᴰ Γᴰ' Δᴰ' τ
    ap-Subᴰᶜ refl refl refl refl refl = refl

    ap-Tyᴰ : A ≡ B → Tyᴰ Γᴰ A ≡ Tyᴰ Γᴰ B
    ap-Tyᴰ refl = refl

    ap-Tyᴰᶜ : (pΓ : Γ ≡ Δ)
      → Γᴰ ≡[ ap-Conᴰᶜ pΓ ] Δᴰ
      → A ≡[ ap-Tyᶜ pΓ ] B
      → Tyᴰ Γᴰ A ≡ Tyᴰ Δᴰ B
    ap-Tyᴰᶜ refl refl refl = refl

    ap-#Єᴰ : π ≡ ξ → #Єᴰ Γᴰ π ≡ #Єᴰ Γᴰ ξ
    ap-#Єᴰ refl = refl

    ap-#Єᴰᶜ : (pΓ : Γ ≡ Δ)
      → Γᴰ ≡[ ap-Conᴰᶜ pΓ ] Δᴰ
      → π ≡[ ap-#Єᶜ pΓ ] ξ
      → #Єᴰ Γᴰ π ≡ #Єᴰ Δᴰ ξ
    ap-#Єᴰᶜ refl refl refl = refl
```

```
  ap-Tmᴰ : (p : A ≡ B)
    → Aᴰ ≡[ ap-Tyᴰ p ] Bᴰ
    → t ≡[ ap-Tm p ] u
    → Tmᴰ Γᴰ i Aᴰ t ≡ Tmᴰ Γᴰ i Bᴰ u
  ap-Tmᴰ refl refl refl = refl

  ap-Tmᴰᶜ : (pΓ : Γ ≡ Δ) → (pA : A ≡[ ap-Tyᶜ pΓ ] B)
    → (pΓᴰ : Γᴰ ≡[ ap-Conᴰᶜ pΓ ] Δᴰ)
    → Aᴰ ≡[ ap-Tyᴰᶜ pΓ pΓᴰ pA ] Bᴰ
    → t ≡[ ap-Tmᶜ pΓ pA ] u
    → Tmᴰ Γᴰ i Aᴰ t ≡ Tmᴰ Δᴰ i Bᴰ u
  ap-Tmᴰᶜ refl refl refl refl refl = refl

  ap-Tmᴰᶜ' : (pΓ : Γ ≡ Δ) → (pA : A ≡[ ap-Tyᶜ pΓ ] B)
    → (pΓᴰ : Γᴰ ≡[ ap-Conᴰᶜ pΓ ] Δᴰ)
    → (pi : i ≡ j)
    → Aᴰ ≡[ ap-Tyᴰᶜ pΓ pΓᴰ pA ] Bᴰ
    → t ≡[ ap-Tmᶜ' pΓ pi pA ] u
    → Tmᴰ Γᴰ i Aᴰ t ≡ Tmᴰ Δᴰ j Bᴰ u
  ap-Tmᴰᶜ' refl refl refl refl refl refl = refl

module core-utilsᴰ
  (_[_]Tᴰ : ∀ {Γ Δ A σ} {Γᴰ : Conᴰ Γ} {Δᴰ : Conᴰ Δ}
    → Tyᴰ Δᴰ A → Subᴰ Γᴰ Δᴰ σ → Tyᴰ Γᴰ (A [ σ ]T))
  where
  opaque
    unfolding ap-Subᴰ ap-Tyᴰ
    ap-[]T₀ᴰ : (p : σ ≡ τ) → σᴰ ≡[ ap-Subᴰ p ] τᴰ
      → Aᴰ [ σᴰ ]Tᴰ ≡[ ap-Tyᴰ (ap-[]T₀ p) ] (Aᴰ [ τᴰ ]Tᴰ)
    ap-[]T₀ᴰ refl refl = refl

    ap-[]T₁ᴰ : (p : A ≡ B) → Aᴰ ≡[ ap-Tyᴰ p ] Bᴰ
      → Aᴰ [ σᴰ ]Tᴰ ≡[ ap-Tyᴰ (ap-[]T₁ p) ] (Bᴰ [ σᴰ ]Tᴰ)
    ap-[]T₁ᴰ refl refl = refl

record CwFwEᴰ-core : Set where
  field
    idᴰ : {Γᴰ : Conᴰ Γ} → Subᴰ Γᴰ Γᴰ id
    _∘ᴰ_ : Subᴰ Δᴰ θᴰ σ → Subᴰ Γᴰ Δᴰ τ → Subᴰ Γᴰ θᴰ (σ ∘ τ)
    assocᴰ : ρᴰ ∘ᴰ (σᴰ ∘ᴰ τᴰ) ≡[ ap-Subᴰ assoc ] (ρᴰ ∘ᴰ σᴰ) ∘ᴰ τᴰ
    id∘ᴰ : idᴰ ∘ᴰ σᴰ ≡[ ap-Subᴰ id∘ ] σᴰ
    ∘idᴰ : σᴰ ∘ᴰ idᴰ ≡[ ap-Subᴰ ∘id ] σᴰ

    •ᴰ : Conᴰ •
    εᴰ : Subᴰ Γᴰ •ᴰ ε
    ∃!εᴰ : εᴰ {Γᴰ = Γᴰ} ≡[ ap-Subᴰ ∃!ε ] σᴰ

    _[_]Tᴰ : Tyᴰ Δᴰ A → Subᴰ Γᴰ Δᴰ σ → Tyᴰ Γᴰ (A [ σ ]T)
    _[_]ᴰ : Tmᴰ Δᴰ i Aᴰ t → (σᴰ : Subᴰ Γᴰ Δᴰ σ) → Tmᴰ Γᴰ i (Aᴰ [ σᴰ ]Tᴰ) (t [ σ ])
    _[_]#ᴰ : #Єᴰ Δᴰ π → Subᴰ Γᴰ Δᴰ σ → #Єᴰ Γᴰ (π [ σ ]#)
    [id]Tᴰ : Aᴰ [ idᴰ ]Tᴰ ≡[ ap-Tyᴰ [id]T ] Aᴰ
    [id]ᴰ : {tᴰ : Tmᴰ Δᴰ i Aᴰ t} → (tᴰ [ idᴰ ]ᴰ) ≡[ ap-Tmᴰ [id]T [id]Tᴰ [id] ] tᴰ
    [id]#ᴰ : πᴰ [ idᴰ ]#ᴰ ≡[ ap-#Єᴰ [id]# ] πᴰ
    [∘]Tᴰ : Aᴰ [ σᴰ ∘ᴰ τᴰ ]Tᴰ ≡[ ap-Tyᴰ [∘]T ] (Aᴰ [ σᴰ ]Tᴰ) [ τᴰ ]Tᴰ
    [∘]ᴰ : {tᴰ : Tmᴰ Δᴰ i Aᴰ t} → (tᴰ [ σᴰ ∘ᴰ τᴰ ]ᴰ) ≡[ ap-Tmᴰ [∘]T [∘]Tᴰ [∘] ] ((tᴰ [ σᴰ ]ᴰ) [ τᴰ ]ᴰ)
    [∘]#ᴰ : πᴰ [ σᴰ ∘ᴰ τᴰ ]#ᴰ ≡[ ap-#Єᴰ [∘]# ] (πᴰ [ σᴰ ]#ᴰ) [ τᴰ ]#ᴰ

  ap-[]T₀ᴰ : (p : σ ≡ τ) → σᴰ ≡[ ap-Subᴰ p ] τᴰ
    → Aᴰ [ σᴰ ]Tᴰ ≡[ ap-Tyᴰ (ap-[]T₀ p) ] Aᴰ [ τᴰ ]Tᴰ
  ap-[]T₀ᴰ = core-utilsᴰ.ap-[]T₀ᴰ _[_]Tᴰ

  ap-[]T₁ᴰ : (p : A ≡ B) → Aᴰ ≡[ ap-Tyᴰ p ] Bᴰ
    → Aᴰ [ σᴰ ]Tᴰ ≡[ ap-Tyᴰ (ap-[]T₁ p) ] Bᴰ [ σᴰ ]Tᴰ
  ap-[]T₁ᴰ = core-utilsᴰ.ap-[]T₁ᴰ _[_]Tᴰ

  field
    _▷ᴰ[_]_ : (Γᴰ : Conᴰ Γ) → (i : Mode) → Tyᴰ Γᴰ A → Conᴰ (Γ ▷[ i ] A)
    pᴰ : Subᴰ (Γᴰ ▷ᴰ[ i ] Aᴰ) Γᴰ p
    qᴰ : Tmᴰ (Γᴰ ▷ᴰ[ i ] Aᴰ) i (Aᴰ [ pᴰ ]Tᴰ) (q {Γ} {i} {A})
    _,,ᴰ_ : (σᴰ : Subᴰ Γᴰ Δᴰ σ) → Tmᴰ Γᴰ i (Aᴰ [ σᴰ ]Tᴰ) t → Subᴰ Γᴰ (Δᴰ ▷ᴰ[ i ] Aᴰ) (σ ,, t)
    ,∘ᴰ : (σᴰ ,,ᴰ tᴰ) ∘ᴰ ρᴰ ≡[ ap-Subᴰ ,∘ ]
        (σᴰ ∘ᴰ ρᴰ) ,,ᴰ coe (ap-Tmᴰ (sym [∘]T) (symᴰ [∘]Tᴰ) refl) (tᴰ [ ρᴰ ]ᴰ)
    p,qᴰ : pᴰ {Γᴰ = Γᴰ} {i = i} {Aᴰ = Aᴰ} ,,ᴰ qᴰ ≡[ ap-Subᴰ p,q ] idᴰ
    p∘,ᴰ : pᴰ ∘ᴰ (σᴰ ,,ᴰ tᴰ) ≡[ ap-Subᴰ p∘, ] σᴰ
    q[,]ᴰ : {tᴰ : Tmᴰ Γᴰ i (Aᴰ [ σᴰ ]Tᴰ) t} → qᴰ [ σᴰ ,,ᴰ tᴰ ]ᴰ
        ≡[ ap-Tmᴰ (trans (sym [∘]T) (ap-[]T₀ p∘,)) ]
```

```
                (transᴰ (symᴰ [∘]Tᴰ) (ap-[]Tₒᴰ p∘, p∘,ᴰ))
                q[,] ]
      tᴰ

  _▷#ᴰ : Conᴰ Γ → Conᴰ (Γ ▷#)
  p#ᴰ : Subᴰ (Γᴰ ▷#ᴰ) Γᴰ p#
  q#ᴰ : #Eᴰ (Γᴰ ▷#ᴰ) q#
  _,#ᴰ_ : Subᴰ Γᴰ Δᴰ σ → #Eᴰ Γᴰ π → Subᴰ Γᴰ (Δᴰ ▷#ᴰ) (σ ,# π)
  ,#∘ᴰ : (σᴰ ,#ᴰ πᴰ) ∘ᴰ ρᴰ ≡[ ap-Subᴰ ,#∘ ] (σᴰ ∘ᴰ ρᴰ) ,#ᴰ (πᴰ [ ρᴰ ]#ᴰ)
  p,#qᴰ : p#ᴰ {Γᴰ = Γᴰ} ,#ᴰ q#ᴰ ≡[ ap-Subᴰ p,#q ] idᴰ
  p∘,#ᴰ : p#ᴰ ∘ᴰ (σᴰ ,#ᴰ πᴰ) ≡[ ap-Subᴰ p∘,# ] σᴰ
  q[,#]ᴰ : q#ᴰ [ σᴰ ,#ᴰ πᴰ ]#ᴰ ≡[ ap-#Eᴰ q[,#] ] πᴰ

⟨_⟩ᴰ : (tᴰ : Tmᴰ Γᴰ i Aᴰ t) → Subᴰ Γᴰ (Γᴰ ▷ᴰ[ i ] Aᴰ) ⟨ t ⟩
⟨ tᴰ ⟩ᴰ = idᴰ ,,ᴰ (tᴰ [ idᴰ ]ᴰ)

_⁺ᴰ : (σᴰ : Subᴰ Γᴰ Δᴰ σ) → Subᴰ (Γᴰ ▷ᴰ[ i ] (Aᴰ [ σᴰ ]Tᴰ)) (Δᴰ ▷ᴰ[ i ] Aᴰ) (σ ⁺)
σᴰ ⁺ᴰ = (σᴰ ∘ᴰ pᴰ) ,,ᴰ coe (ap-Tmᴰ (sym [∘]T) (symᴰ [∘]Tᴰ) refl) qᴰ

_⁺#ᴰ : (σᴰ : Subᴰ Γᴰ Δᴰ σ) → Subᴰ (Γᴰ ▷#ᴰ) (Δᴰ ▷#ᴰ) (σ ⁺#)
σᴰ ⁺#ᴰ = (σᴰ ∘ᴰ p#ᴰ) ,#ᴰ q#ᴰ

field
  ↓ᴰ : Tmᴰ {Γ ▷#} (Γᴰ ▷#ᴰ) ω (Aᴰ [ p#ᴰ ]Tᴰ) t → Tmᴰ Γᴰ z Aᴰ (↓ t)
  ↑ᴰ : Tmᴰ Γᴰ z Aᴰ t → Tmᴰ (Γᴰ ▷#ᴰ) ω (Aᴰ [ p#ᴰ ]Tᴰ) (↑ t)
  ↓[]ᴰ : (↓ᴰ tᴰ) [ σᴰ ]ᴰ ≡[ ap-Tmᴰ refl reflᴰ (dep ↓[]) ]
    ↓ᴰ (coe (ap-Tmᴰ (trans (sym [∘]T) (trans (ap-[]Tₒ p∘,#) [∘]T))
              (transᴰ (symᴰ [∘]Tᴰ)
              (transᴰ (ap-[]Tₒᴰ p∘,# p∘,#ᴰ) [∘]Tᴰ)) refl)
              (tᴰ [ σᴰ ⁺#ᴰ ]ᴰ))
  ↑↓ᴰ : ↑ᴰ (↓ᴰ tᴰ) ≡[ ap-Tmᴰ refl reflᴰ (dep ↑↓) ] tᴰ
  ↓↑ᴰ : ↓ᴰ (↑ᴰ tᴰ) ≡[ ap-Tmᴰ refl reflᴰ (dep ↓↑) ] tᴰ

opaque
  unfolding ↓*  pz

  ↓*ᴰ : Tmᴰ Γᴰ i Aᴰ t → Tmᴰ Γᴰ z Aᴰ (↓* t)
  ↓*ᴰ {i = z} tᴰ = tᴰ
  ↓*ᴰ {i = ω} tᴰ = ↓ᴰ (tᴰ [ p#ᴰ ]ᴰ)

  pzᴰ : Subᴰ (Γᴰ ▷ᴰ[ i ] Aᴰ) (Γᴰ ▷ᴰ[ z ] Aᴰ) pz
  pzᴰ = pᴰ ,,ᴰ ↓*ᴰ qᴰ

  p,↓*qᴰ : (pᴰ {Γᴰ = Γᴰ} {i = _} {Aᴰ = Aᴰ} ,,ᴰ ↓*ᴰ {i = z} qᴰ) ≡[ ap-Subᴰ p,↓*q ] idᴰ
  p,↓*qᴰ = p,qᴰ

module in-CwFwEᴰ-core (cᴰ : CwFwEᴰ-core) where
  open CwFwEᴰ-core cᴰ

  record Π-structureᴰ (ps : Π-structure) : Set where
    open Π-structure ps
    field
      Πᴰ : (i : Mode) → (Aᴰ : Tyᴰ Γᴰ A) → (Bᴰ : Tyᴰ (Γᴰ ▷ᴰ[ z ] Aᴰ) B) → Tyᴰ Γᴰ (Π i A B)
      Π[]ᴰ : (Πᴰ i Aᴰ Bᴰ) [ σᴰ ]Tᴰ ≡[ ap-Tyᴰ Π[] ] Πᴰ i (Aᴰ [ σᴰ ]Tᴰ) (Bᴰ [ σᴰ ⁺ᴰ ]Tᴰ)

      lamᴰ : (tᴰ : Tmᴰ (Γᴰ ▷ᴰ[ i ] Aᴰ) ω (Bᴰ [ pzᴰ ]Tᴰ) t) → Tmᴰ Γᴰ ω (Πᴰ i Aᴰ Bᴰ) (lam t)
      -- Not including lam[]ᴰ because that would require proving the lemma pz∘⁺≡⁺∘pz
      -- in the displayed case. This doesn't even matter when we eliminate into Prop
      -- which is always the case in this repo. Either way it is possible to prove it.

      apᴰ : (tᴰ : Tmᴰ Γᴰ ω (Πᴰ i Aᴰ Bᴰ) t) → Tmᴰ (Γᴰ ▷ᴰ[ i ] Aᴰ) ω (Bᴰ [ pzᴰ ]Tᴰ) (ap t)

      Πβᴰ : apᴰ {i = i} (lamᴰ tᴰ) ≡[ ap-Tmᴰ refl reflᴰ (dep Πβ) ] tᴰ
      Πηᴰ : lamᴰ {i = i} (apᴰ tᴰ) ≡[ ap-Tmᴰ refl reflᴰ (dep Πη) ] tᴰ

  record U-structureᴰ (us : U-structure) : Set where
    open U-structure us
    field
      Uᴰ : Tyᴰ Γᴰ U
      U[]ᴰ : Uᴰ [ σᴰ ]Tᴰ ≡[ ap-Tyᴰ U[] ] Uᴰ

      Elᴰ : (tᴰ : Tmᴰ Γᴰ z Uᴰ t) → Tyᴰ Γᴰ (El t)
      El[]ᴰ : (Elᴰ tᴰ) [ σᴰ ]Tᴰ ≡[ ap-Tyᴰ El[] ] Elᴰ (coe (ap-Tmᴰ U[] U[]ᴰ refl) (tᴰ [ σᴰ ]ᴰ))

      codeᴰ : (Aᴰ : Tyᴰ Γᴰ A) → Tmᴰ Γᴰ z Uᴰ (code A)
      code[]ᴰ : (codeᴰ Aᴰ) [ σᴰ ]ᴰ ≡[ ap-Tmᴰ U[] U[]ᴰ code[] ] codeᴰ (Aᴰ [ σᴰ ]Tᴰ)
```

```
        El-codeᴰ : Elᴰ (codeᴰ Aᴰ) ≡[ ap-Tyᴰ El-code ] Aᴰ
        code-Elᴰ : codeᴰ (Elᴰ tᴰ) ≡[ ap-Tmᴰ refl reflᴰ (dep code-El) ] tᴰ

record CwFwEᴰ (base : CwFwE) : Set where
  open CwFwE base
  field
    sortsᴰ : CwFwEᴰ-sorts sorts
  open in-CwFwEᴰ-sorts sortsᴰ core
  field
    coreᴰ : CwFwEᴰ-core
  open in-CwFwEᴰ-core coreᴰ
  field
    Π-strᴰ : Π-structureᴰ Π-str
    U-strᴰ : U-structureᴰ U-str


-- Syntax

module CwFwE-syntax where
  postulate
    syn : CwFwE

  open CwFwE syn public
  open CwFwE-sorts sorts public
  open in-CwFwE-sorts sorts public
  open in-CwFwE-sorts.CwFwE-core core public
  open in-CwFwE-sorts.in-CwFwE-core sorts core public
  open in-CwFwE-sorts.in-CwFwE-core.U-structure U-str public
  open in-CwFwE-sorts.in-CwFwE-core.Π-structure Π-str public

  -- Injectivity for syntax sorts

  postulate
    Sub-injₒ : Sub Γ Δ ≡ Sub Γ' Δ' → Γ ≡ Γ'

    Sub-injₗ : Sub Γ Δ ≡ Sub Γ' Δ' → Δ ≡ Δ'

    Ty-inj : Ty Γ ≡ Ty Γ' → Γ ≡ Γ'

    #∈-inj : #∈ Γ ≡ #∈ Γ' → Γ ≡ Γ'

    Tm-injᶜ : Tm Γ i A ≡ Tm Γ' j A' → Γ ≡ Γ'
    Tm-injᵀ : (e : Tm Γ i A ≡ Tm Γ' j A') → A ≡[ ap-Tyᶜ (Tm-injᶜ e) ] A'
    Tm-injⁱ : (e : Tm Γ i A ≡ Tm Γ' j A') → i ≡ j

  -- Eliminator
  module CwFwE-elim (methods : CwFwEᴰ syn) where
    open CwFwEᴰ methods
    open CwFwEᴰ-sorts sortsᴰ
    open in-CwFwEᴰ-sorts sortsᴰ core
    open in-CwFwEᴰ-sorts.CwFwEᴰ-core coreᴰ
    open in-CwFwEᴰ-sorts.in-CwFwEᴰ-core sortsᴰ core coreᴰ

    postulate
       ⟦_⟧ᶜ : (Γ : Con) → Conᴰ Γ
       ⟦_⟧ˢ : (σ : Sub Γ Δ) → Subᴰ ⟦ Γ ⟧ᶜ ⟦ Δ ⟧ᶜ σ
       ⟦_⟧ᵀ : (A : Ty Γ) → Tyᴰ ⟦ Γ ⟧ᶜ A
       ⟦_⟧# : (π : #∈ Γ) → #∈ᴰ ⟦ Γ ⟧ᶜ π
       ⟦_⟧ᵗ : (a : Tm Γ i A) → Tmᴰ ⟦ Γ ⟧ᶜ i ⟦ A ⟧ᵀ a

    opaque
      unfolding coe

      ap-⟦⟧ᶜ : (e : Γ ≡ Γ') → ⟦ Γ ⟧ᶜ ≡[ ap-Conᴰᶜ e ] ⟦ Γ' ⟧ᶜ
      ap-⟦⟧ᶜ refl = refl

      ap-⟦⟧ᵀ : (eΓ : Γ ≡ Γ') → (eA : A ≡[ ap-Tyᶜ eΓ ] A')
        → ⟦ A ⟧ᵀ ≡[ ap-Tyᴰᶜ eΓ (ap-⟦⟧ᶜ eΓ) eA ] ⟦ A' ⟧ᵀ
      ap-⟦⟧ᵀ refl refl = refl

    postulate
       ⟦coe⟧ˢ : ∀ (p : Sub Γ Δ ≡ Sub Γ' Δ') →
        ⟦ coe p σ ⟧ˢ
          ≡ coe (ap-Subᴰᶜ (Sub-injₒ p) (Sub-injₗ p) (ap-⟦⟧ᶜ (Sub-injₒ p)) (ap-⟦⟧ᶜ (Sub-injₗ p)) refl ) ⟦ σ ⟧ˢ
       {-# REWRITE ⟦coe⟧ˢ #-}
```

```
  ⟦coe⟧ᵀ : ∀ (p : Ty Γ ≡ Ty Γ') →
   ⟦ coe p A ⟧ᵀ
     ≡ coe (ap-Tyᴰᶜ (Ty-inj p) (ap-⟦⟧ᶜ (Ty-inj p)) refl) ⟦ A ⟧ᵀ
  {-# REWRITE ⟦coe⟧ᵀ #-}

  ⟦coe⟧# : ∀ (p : #∈ Γ ≡ #∈ Γ') →
   ⟦ coe p π ⟧#
     ≡ coe (ap-#∈ᴰᶜ (#∈-inj p) (ap-⟦⟧ᶜ (#∈-inj p)) refl) ⟦ π ⟧#
  {-# REWRITE ⟦coe⟧# #-}

  ⟦coe⟧ᵗ : ∀ (p : Tm Γ i A ≡ Tm Γ' j A') →
   ⟦ coe p t ⟧ᵗ
     ≡ coe (ap-Tmᴰᶜ' (Tm-injᶜ p)  (Tm-injᵀ p) (ap-⟦⟧ᶜ (Tm-injᶜ p)) (Tm-injⁱ p) (ap-⟦⟧ᵀ (Tm-injᶜ p)
     ↪  (Tm-injᵀ p)) refl) ⟦ t ⟧ᵗ
  {-# REWRITE ⟦coe⟧ᵗ #-}

postulate
  ⟦•⟧ : ⟦ • ⟧ᶜ ≡ •ᴰ
  {-# REWRITE ⟦•⟧ #-}

  ⟦id⟧ : ⟦ id {Γ} ⟧ˢ ≡ idᴰ
  {-# REWRITE ⟦id⟧ #-}

  ⟦∘⟧ : ⟦ σ ∘ τ ⟧ˢ ≡ ⟦ σ ⟧ˢ ∘ᴰ ⟦ τ ⟧ˢ
  {-# REWRITE ⟦∘⟧ #-}

  ⟦ε⟧ : ⟦ ε {Γ} ⟧ˢ ≡ εᴰ
  {-# REWRITE ⟦ε⟧ #-}

  ⟦▷⟧ : ⟦ Γ ▷[ i ] A ⟧ᶜ ≡ ⟦ Γ ⟧ᶜ ▷ᴰ[ i ] ⟦ A ⟧ᵀ
  {-# REWRITE ⟦▷⟧ #-}

  ⟦▷#⟧ : ⟦ Γ ▷# ⟧ᶜ ≡ ⟦ Γ ⟧ᶜ ▷#ᴰ
  {-# REWRITE ⟦▷#⟧ #-}

  ⟦[]T⟧ : ⟦ A [ σ ]T ⟧ᵀ ≡ ⟦ A ⟧ᵀ [ ⟦ σ ⟧ˢ ]Tᴰ
  {-# REWRITE ⟦[]T⟧ #-}

  ⟦p⟧ : ⟦ p {Γ} {i} {A} ⟧ˢ ≡ pᴰ
  {-# REWRITE ⟦p⟧ #-}

  ⟦p#⟧ : ⟦ p# {Γ} ⟧ˢ ≡ p#ᴰ
  {-# REWRITE ⟦p#⟧ #-}

  ⟦[]⟧ : ⟦ t [ σ ] ⟧ᵗ ≡ (⟦ t ⟧ᵗ [ ⟦ σ ⟧ˢ ]ᴰ)
  {-# REWRITE ⟦[]⟧ #-}

  ⟦[]#⟧ : ⟦ π [ σ ]# ⟧# ≡ ⟦ π ⟧# [ ⟦ σ ⟧ˢ ]#ᴰ
  {-# REWRITE ⟦[]#⟧ #-}

  ⟦q⟧ : ⟦ q {Γ} {i} {A} ⟧ᵗ ≡ qᴰ
  {-# REWRITE ⟦q⟧ #-}

  ⟦,,⟧ : ⟦ σ ,, t ⟧ˢ ≡ ⟦ σ ⟧ˢ ,,ᴰ ⟦ t ⟧ᵗ
  {-# REWRITE ⟦,,⟧ #-}

  ⟦q#⟧ : ⟦ q# {Γ} ⟧# ≡ q#ᴰ
  {-# REWRITE ⟦q#⟧ #-}

  ⟦,#⟧ : ⟦ σ ,# π ⟧ˢ ≡ ⟦ σ ⟧ˢ ,#ᴰ ⟦ π ⟧#
  {-# REWRITE ⟦,#⟧ #-}

  ⟦↓⟧ : ⟦ ↓ t ⟧ᵗ ≡ ↓ᴰ ⟦ t ⟧ᵗ
  {-# REWRITE ⟦↓⟧ #-}

  ⟦↑⟧ : ⟦ ↑ t ⟧ᵗ ≡ ↑ᴰ ⟦ t ⟧ᵗ
  {-# REWRITE ⟦↑⟧ #-}

opaque
  unfolding ↓*ᴰ

  ⟦↓*⟧ : {t : Tm Γ i A} → ⟦ ↓* t ⟧ᵗ ≡ ↓*ᴰ ⟦ t ⟧ᵗ
  ⟦↓*⟧ {i = z} = refl
  ⟦↓*⟧ {i = ω} = refl
```

```
    {-# REWRITE ⟦↓*⟧ #-}

    opaque
      unfolding pzᴰ

      ⟦pz⟧ : ⟦ pz {Γ} {i} {A} ⟧ˢ ≡ pzᴰ
      ⟦pz⟧ {i = z} = refl
      ⟦pz⟧ {i = ω} = refl

    {-# REWRITE ⟦pz⟧ #-}

    open Π-structureᴰ Π-strᴰ
    open U-structureᴰ U-strᴰ

    postulate
       ⟦Π⟧ : ⟦ Π i A B ⟧ᵀ ≡ Πᴰ i ⟦ A ⟧ᵀ ⟦ B ⟧ᵀ
       {-# REWRITE ⟦Π⟧ #-}

       ⟦lam⟧ : ⟦ lam {i = i} t ⟧ᵗ ≡ lamᴰ ⟦ t ⟧ᵗ
       {-# REWRITE ⟦lam⟧ #-}

       ⟦ap⟧ : ⟦ ap {i = i} t ⟧ᵗ ≡ apᴰ ⟦ t ⟧ᵗ
       {-# REWRITE ⟦ap⟧ #-}

       ⟦U⟧ : ⟦ U {Γ} ⟧ᵀ ≡ Uᴰ
       {-# REWRITE ⟦U⟧ #-}

       ⟦El⟧ : ⟦ El t ⟧ᵀ ≡ Elᴰ ⟦ t ⟧ᵗ
       {-# REWRITE ⟦El⟧ #-}

       ⟦code⟧ : ⟦ code A ⟧ᵗ ≡ codeᴰ ⟦ A ⟧ᵀ
       {-# REWRITE ⟦code⟧ #-}

-- Context eliminator
module CwFwE-elim-Con
  (open CwFwE-syntax)
  (P : Con → Set)
  (P• : P •)
  (P▷[z] : ∀ Γ A → P Γ → P (Γ ▷[ z ] A))
  (P▷[ω] : ∀ Γ A → P Γ → P (Γ ▷[ ω ] A))
  (P▷# : ∀ Γ → P Γ → P (Γ ▷#))
  where

  open CwFwEᴰ
  open CwFwEᴰ-sorts
  open in-CwFwEᴰ-sorts
  open CwFwEᴰ-core
  open in-CwFwEᴰ-core
  open Π-structureᴰ
  open U-structureᴰ

  nᴰ-sorts : CwFwEᴰ-sorts sorts
  nᴰ-sorts .Conᴰ Γ = P Γ
  nᴰ-sorts .Subᴰ _ _ _ = 𝟙
  nᴰ-sorts .Tyᴰ _ _ = 𝟙
  nᴰ-sorts .#Cᴰ _ _ = 𝟙
  nᴰ-sorts .Tmᴰ _ _ _ _ = 𝟙
  nᴰ-sorts .#-propᴰ p₁ q₁ prf = refl

  nᴰ-core : CwFwEᴰ-core nᴰ-sorts core
  nᴰ-core .idᴰ = tt
  nᴰ-core ._∘ᴰ_ = λ _ _ → tt
  nᴰ-core .assocᴰ = refl
  nᴰ-core .id∘ᴰ = refl
  nᴰ-core .∘idᴰ = refl
  nᴰ-core .•ᴰ = P•
  nᴰ-core .εᴰ = tt
  nᴰ-core .∃!εᴰ = refl
  nᴰ-core ._[_]Tᴰ = λ _ _ → tt
  nᴰ-core ._[_]ᴰ = λ _ σᴰ → tt
  nᴰ-core ._[_]#ᴰ = λ _ _ → tt
  nᴰ-core .[id]Tᴰ = refl
  nᴰ-core .[id]ᴰ = refl
  nᴰ-core .[id]#ᴰ = refl
  nᴰ-core .[∘]Tᴰ = refl
  nᴰ-core .[∘]ᴰ = refl
```

```
  nᴰ-core .[∘]#ᴰ = refl
  _▷ᴰ[_]_ nᴰ-core {Γ = Γ} {A = A} Γᴰ z _ = Pᴰ▷[z] Γ A Γᴰ
  _▷ᴰ[_]_ nᴰ-core {Γ = Γ} {A = A} Γᴰ ω _ = Pᴰ▷[ω] Γ A Γᴰ
  nᴰ-core .pᴰ = tt
  nᴰ-core .qᴰ = tt
  nᴰ-core ._,,ᴰ_ = λ σᴰ _ → tt
  nᴰ-core .,∘ᴰ = refl
  nᴰ-core .p,qᴰ = refl
  nᴰ-core .p∘,ᴰ = refl
  nᴰ-core .q[,]ᴰ = refl
  nᴰ-core ._▷#ᴰ {Γ = Γ} = Pᴰ▷# Γ
  nᴰ-core .p#ᴰ = tt
  nᴰ-core .q#ᴰ = tt
  nᴰ-core ._,#ᴰ_ = λ _ _ → tt
  nᴰ-core .,#∘ᴰ = refl
  nᴰ-core .p,#qᴰ = refl
  nᴰ-core .p∘,#ᴰ = refl
  nᴰ-core .q[,#]ᴰ = refl
  nᴰ-core .↓ᴰ = λ _ → tt
  nᴰ-core .↑ᴰ = λ _ → tt
  nᴰ-core .↓[]ᴰ = refl
  nᴰ-core .↑↓ᴰ = refl
  nᴰ-core .↓↑ᴰ = refl

  nᴰ-Π-str : Π-structureᴰ nᴰ-sorts core nᴰ-core Π-str
  nᴰ-Π-str .Πᴰ = λ i Aᴰ Bᴰ → tt
  nᴰ-Π-str .Π[]ᴰ = refl
  nᴰ-Π-str .lamᴰ = λ tᴰ → tt
  nᴰ-Π-str .apᴰ = λ tᴰ → tt
  nᴰ-Π-str .Πβᴰ = refl
  nᴰ-Π-str .Πηᴰ = refl

  nᴰ-U-str : U-structureᴰ nᴰ-sorts core nᴰ-core U-str
  nᴰ-U-str .Uᴰ = tt
  nᴰ-U-str .U[]ᴰ = refl
  nᴰ-U-str .Elᴰ = λ tᴰ → tt
  nᴰ-U-str .El[]ᴰ = refl
  nᴰ-U-str .codeᴰ = λ Aᴰ → tt
  nᴰ-U-str .code[]ᴰ = refl
  nᴰ-U-str .El-codeᴰ = refl
  nᴰ-U-str .code-Elᴰ = refl

  nᴰ : CwFwEᴰ syn
  nᴰ .CwFwEᴰ.sortsᴰ = nᴰ-sorts
  nᴰ .CwFwEᴰ.coreᴰ = nᴰ-core
  nᴰ .CwFwEᴰ.Π-strᴰ = nᴰ-Π-str
  nᴰ .CwFwEᴰ.U-strᴰ = nᴰ-U-str

  ⟦_⟧ : (Γ : Con) → P Γ
  ⟦_⟧ Γ = CwFwE-elim.⟦_⟧ᶜ nᴰ Γ


module CwFwE-uniform (m : CwFwE) (n : CwFwE) where

  open CwFwE n
  open CwFwE-sorts sorts
  open in-CwFwE-sorts sorts
  open CwFwE-core core
  open in-CwFwE-core core
  open Π-structure Π-str
  open U-structure U-str

  open CwFwEᴰ
  open CwFwEᴰ-sorts
  open in-CwFwEᴰ-sorts
  open CwFwEᴰ-core
  open in-CwFwEᴰ-core
  open Π-structureᴰ
  open U-structureᴰ

  nᴰ-sorts : CwFwEᴰ-sorts (m .CwFwE.sorts)
  nᴰ-sorts .Conᴰ _ = Con
  nᴰ-sorts .Subᴰ Γ Δ _ = Sub Γ Δ
  nᴰ-sorts .Tyᴰ Γ _ = Ty Γ
  nᴰ-sorts .#Єᴰ Γ _ = #Є Γ
  nᴰ-sorts .Tmᴰ Γ i A _ = Tm Γ i A
```

```
nᴰ-sorts .#-propᴰ p₁ q₁ prf = #-prop (coe _ p₁) q₁

nᴰ-core : CwFwEᴰ-core nᴰ-sorts (m .CwFwE.core)
nᴰ-core .idᴰ = id
nᴰ-core ._∘ᴰ_ = _∘_
nᴰ-core .assocᴰ = dep assoc
nᴰ-core .∘idᴰ = dep ∘id
nᴰ-core .id∘ᴰ = dep id∘
nᴰ-core .•ᴰ = •
nᴰ-core .εᴰ = ε
nᴰ-core .∃!εᴰ = dep ∃!ε
nᴰ-core ._[_]Tᴰ =  _[_]T
nᴰ-core ._[_]ᴰ = _[_]
nᴰ-core ._[_]#ᴰ = _[_]#
nᴰ-core .[id]Tᴰ = dep [id]T
nᴰ-core .[id]ᴰ = [id]
nᴰ-core .[id]#ᴰ = dep [id]#
nᴰ-core .[∘]Tᴰ = dep [∘]T
nᴰ-core .[∘]ᴰ = [∘]
nᴰ-core .[∘]#ᴰ = dep [∘]#
nᴰ-core ._▷ᴰ[_]_ = _▷[_]_
nᴰ-core .pᴰ = p
nᴰ-core .qᴰ = q
nᴰ-core ._,,ᴰ_ = _,,_
nᴰ-core .,∘ᴰ = dep ,∘
nᴰ-core .p,qᴰ = dep p,q
nᴰ-core .p∘,ᴰ = dep p∘,
nᴰ-core .q[,]ᴰ = q[,]
nᴰ-core ._▷#ᴰ = _▷#
nᴰ-core .p#ᴰ = p#
nᴰ-core .q#ᴰ = q#
nᴰ-core ._,#ᴰ_ = _,#_
nᴰ-core .,#∘ᴰ = dep ,#∘
nᴰ-core .p,#qᴰ = dep p,#q
nᴰ-core .p∘,#ᴰ = dep p∘,#
nᴰ-core .q[,#]ᴰ = dep q[,#]
nᴰ-core .↓ᴰ = ↓
nᴰ-core .↑ᴰ = ↑
nᴰ-core .↓[]ᴰ = dep ↓[]
nᴰ-core .↑↓ᴰ = dep ↑↓
nᴰ-core .↓↑ᴰ = dep ↓↑

opaque
  unfolding pzᴰ ↓*ᴰ

  nᴰ-Π-str : Π-structureᴰ nᴰ-sorts (m .CwFwE.core) nᴰ-core (m .CwFwE.Π-str)
  nᴰ-Π-str .Πᴰ = Π
  nᴰ-Π-str .Π[]ᴰ = dep Π[]
  nᴰ-Π-str .lamᴰ {i = z} =  lam {i = z}
  nᴰ-Π-str .lamᴰ {i = ω} =  lam {i = ω}
  nᴰ-Π-str .apᴰ {i = z} = ap
  nᴰ-Π-str .Πβᴰ {i = z} = dep Πβ
  nᴰ-Π-str .Πηᴰ {i = z} = dep Πη
  nᴰ-Π-str .apᴰ {i = ω} = ap
  nᴰ-Π-str .Πβᴰ {i = ω} = dep Πβ
  nᴰ-Π-str .Πηᴰ {i = ω} = dep Πη

nᴰ-U-str : U-structureᴰ nᴰ-sorts (m .CwFwE.core) nᴰ-core (m .CwFwE.U-str)
nᴰ-U-str .Uᴰ = U
nᴰ-U-str .U[]ᴰ = dep U[]
nᴰ-U-str .Elᴰ = El
nᴰ-U-str .El[]ᴰ = dep El[]
nᴰ-U-str .codeᴰ = code
nᴰ-U-str .code[]ᴰ = code[]
nᴰ-U-str .El-codeᴰ = dep El-code
nᴰ-U-str .code-Elᴰ = dep code-El

nᴰ : CwFwEᴰ m
nᴰ .CwFwEᴰ.sortsᴰ = nᴰ-sorts
nᴰ .CwFwEᴰ.coreᴰ = nᴰ-core
nᴰ .CwFwEᴰ.Π-strᴰ = nᴰ-Π-str
nᴰ .CwFwEᴰ.U-strᴰ = nᴰ-U-str
```

Models/FO/ExtractionCorrectness.agda

```
{-# OPTIONS --type-in-type #-}
module Models.FO.ExtractionCorrectness where

open import Data.Product
open import Data.Product.Properties
open import Data.Unit
open import Data.Empty
open import Data.Nat hiding (zero)
open import Data.Vec

open import Utils hiding (⊤; ⊥)
open import Theories.CwFwE
open import Mode

-- The code extraction model
--
-- This is a CwFwE containing untyped lambda calculus, the standard
-- interpretation, as well as a logical relation between the two.
--
-- We can thus derive correctness properties about the extracted code (see
-- bottom of file)
--
-- Morally, this whole file lives in Psh(Λ), which is why we have a second-order
-- model of Λ available---the representables. We can take the global sections of
-- theorems to get the corresponding statement in Set about closed Λ terms.
-- This is easier than to formalise in Set because then we would need to deal with
-- closed first-order Λ terms.
--
-- It is also possible to separate the base interpretations from the logical relation
-- through a displayed model, but this seems easier.

postulate
  Λ : Set
  ƛ : (Λ → Λ) → Λ
  _@_ : Λ → Λ → Λ
  Λη : ∀ f →  ƛ (f @_) ≡ f
  Λβ : ∀ f x → (ƛ f) @ x ≡ f x
  {-# REWRITE Λη Λβ #-}

  zeroΛ : Λ
  succΛ : Λ → Λ
  recΛ : Λ → (Λ → Λ → Λ) → Λ → Λ
  recΛβ-zero : ∀ {zr su} → recΛ zr su zeroΛ ≡ zr
  recΛβ-succ : ∀ {zr su n} → recΛ zr su (succΛ n) ≡ su n (recΛ zr su n)
  {-# REWRITE recΛβ-zero recΛβ-succ #-}

  zeroΛ≠succΛ : ∀ {x} → zeroΛ ≡ succΛ x → ⊥
{-# INJECTIVE succΛ #-}

succΛ-inj : ∀ {x y} → succΛ x ≡ succΛ y → x ≡ y
succΛ-inj refl = refl

embed-nat : ℕ → Λ
embed-nat ℕ.zero = zeroΛ
embed-nat (suc x) = succΛ (embed-nat x)

embed-nat-inj : ∀ n m → embed-nat n ≡ embed-nat m → n ≡ m
embed-nat-inj ℕ.zero ℕ.zero p = refl
embed-nat-inj ℕ.zero (suc m) p = exfalso-empty-prop (zeroΛ≠succΛ p)
embed-nat-inj (suc n) ℕ.zero p = exfalso-empty-prop (zeroΛ≠succΛ (sym p))
embed-nat-inj (suc n) (suc m) p = cong suc (embed-nat-inj _ _ (succΛ-inj p))

module _ where
  open CwFwE-sorts
  open in-CwFwE-sorts
  open CwFwE-core
  open in-CwFwE-core

  opaque
    unfolding coe pz

    fam-s : CwFwE-sorts
    fam-s .Con =
      Σ[ Γₛ ∈ Set ]
      Σ[ Γₑ ∈ Set ]
```

```
  Σ[ Γ₀ ∈ (Γₛ → Set) ]
  ((γₛ : Γₛ) → Γₑ → Γ₀ γₛ → Set)
fam-s .Sub (Γₛ , Γₑ , Γ₀ , Γ₁) (Δₛ , Δₑ , Δ₀ , Δ₁) =
  Σ[ σₛ ∈ (Γₛ → Δₛ) ]
  Σ[ σₑ ∈ (Γₑ → Δₑ) ]
  Σ[ σ₀ ∈ (∀ γₛ → Γ₀ γₛ → Δ₀ (σₛ γₛ)) ]
  (∀ γₛ γₑ γ → Γ₁ γₛ γₑ γ → Δ₁ (σₛ γₛ) (σₑ γₑ) (σ₀ γₛ γ))
fam-s .Ty (Γₛ , Γₑ , Γ₀ , Γ₁) =
  Σ[ Aₛ ∈ (Γₛ → Set) ]
  Σ[ A₀ ∈ (∀ γₛ → Γ₀ γₛ → Aₛ γₛ → Set) ]
  (∀ γₛ γ → (aₛ : Aₛ γₛ) → Λ → A₀ γₛ γ aₛ → Set)
fam-s .Tm (Γₛ , Γₑ , Γ₀ , Γ₁) ω (Aₛ , A₀ , A₁) =
  Σ[ aₛ ∈ ((γ : Γₛ) → Aₛ γ) ]
  Σ[ aₑ ∈ (Γₑ → Λ) ]
  Σ[ a₀ ∈ (∀ γₛ → (γ : Γ₀ γₛ) → A₀ γₛ γ (aₛ γₛ)) ]
  (∀ γₛ γₑ γ → (γ' : Γ₁ γₛ γₑ γ) → A₁ γₛ γ (aₛ γₛ) (aₑ γₑ) (a₀ γₛ γ))
fam-s .Tm (Γₛ , Γₑ , Γ₀ , Γ₁) z (Aₛ , A₀ , A₁) =
  Σ[ aₛ ∈ ((γₛ : Γₛ) → Aₛ γₛ) ]
  (∀ γₛ → (γ : Γ₀ γₛ) → A₀ γₛ γ (aₛ γₛ))
fam-s .#∈ (Γₛ , Γₑ , Γ₀ , Γ₁) = Γₑ → ⊥
fam-s .#-prop p₁ q₁ = refl

fam-c : CwFwE-core fam-s
fam-c .id = (λ z₁ → z₁) , (λ z₁ → z₁) , (λ γₛ z₁ → z₁) , (λ γₛ γₑ γ z₁ → z₁)
(fam-c ∘ (σₛ , σₑ , σ₀ , σ₁)) (τₛ , τₑ , τ₀ , τ₁) =
  (λ z₁ → σₛ (τₛ z₁)) ,
  (λ z₁ → σₑ (τₑ z₁)) ,
  (λ γₛ z₁ → σ₀ (τₛ γₛ) (τ₀ γₛ z₁)) ,
  (λ γₛ γₑ γ z₁ → σ₁ (τₛ γₛ) (τₑ γₑ) (τ₀ γₛ γ) (τ₁ γₛ γₑ γ z₁))
fam-c .assoc = refl
fam-c .∘id = refl
fam-c .id∘ = refl
fam-c .• = ⊤ , ⊤ , (λ _ → ⊤) , (λ _ _ _ → ⊤)
fam-c .ε = (λ _ → tt) , (λ _ → tt) , (λ γₛ _ → tt) , (λ γₛ γₑ γ _ → tt)
fam-c .∃!ε = refl
(fam-c [ Aₛ , A₀ , A₁ ]T) (σₛ , σₑ , σ₀ , σ₁) =
  (λ γₛ → Aₛ (σₛ γₛ))
  , (λ γₛ γ → A₀ (σₛ γₛ) (σ₀ γₛ γ))
  , (λ γₛ γ aₛ a → A₁ (σₛ γₛ) (σ₀ γₛ γ) aₛ a)
fam-c ._[_] {i = ω} (tₛ , tₑ , t₀ , t₁) (σₛ , σₑ , σ₀ , σ₁) =
  (λ γ → tₛ (σₛ γ)) ,
    (λ z₁ → tₑ (σₑ z₁)) ,
    (λ γₛ γ → t₀ (σₛ γₛ) (σ₀ γₛ γ)) ,
    (λ γₛ γₑ γ γ' → t₁ (σₛ γₛ) (σₑ γₑ) (σ₀ γₛ γ) (σ₁ γₛ γₑ γ γ'))
fam-c ._[_] {i = z} (tₛ , t₀) (σₛ , σₑ , σ₀ , σ₁) =
  (λ γₛ → tₛ (σₛ γₛ)) , (λ γₛ γ → t₀ (σₛ γₛ) (σ₀ γₛ γ))
fam-c .[id]T = refl
fam-c .[id] {i = z} = refl
fam-c .[id] {i = ω} = refl
fam-c .[∘]T = refl
fam-c .[∘] {i = z} = refl
fam-c .[∘] {i = ω} = refl
(fam-c ▷[ (Γₛ , Γₑ , Γ₀ , Γ₁) ] ω) (Aₛ , A₀ , A₁) =
  (Σ[ γₛ ∈ Γₛ ] Aₛ γₛ)
  , (Γₑ × Λ)
  , (λ (γₛ , aₛ) → Σ[ γ ∈ Γ₀ γₛ ] A₀ γₛ γ aₛ)
  , λ (γₛ , aₛ) (γₑ , aₑ) (γ , a) → Σ[ γ' ∈ Γ₁ γₛ γₑ γ ] A₁ γₛ γ aₛ aₑ a
(fam-c ▷[ (Γₛ , Γₑ , Γ₀ , Γ₁) ] z) (Aₛ , A₀ , A₁) =
  (Σ[ γₛ ∈ Γₛ ] Aₛ γₛ)
  , Γₑ
  , (λ (γₛ , aₛ) → Σ[ γ ∈ Γ₀ γₛ ] A₀ γₛ γ aₛ)
  , λ (γₛ , aₛ) γₑ (γ , a) → Γ₁ γₛ γₑ γ
fam-c .p {i = ω} =
  (λ z₁ → z₁ .proj₁) ,
  (λ z₁ → z₁ .proj₁) ,
  (λ γₛ z₁ → z₁ .proj₁) , (λ γₛ γₑ γ z₁ → z₁ .proj₁)
fam-c .p {i = z} =
  (λ z₁ → z₁ .proj₁) ,
  (λ z₁ → z₁) , (λ γₛ z₁ → z₁ .proj₁) , (λ γₛ γₑ γ z₁ → z₁)
fam-c .q {i = ω} =
  (λ γ → γ .proj₂) ,
  (λ z₁ → z₁ .proj₂) ,
  (λ γₛ γ → γ .proj₂) , (λ γₛ γₑ γ γ' → γ' .proj₂)
fam-c .q {i = z} =
  (λ γₛ → γₛ .proj₂) , (λ γₛ γ → γ .proj₂)
fam-c ._,,_ {i = ω} (σₛ , σₑ , σ₀ , σ₁) (tₛ , tₑ , t₀ , t₁)
```

```
  = (λ z₁ → σₛ z₁ , tₛ z₁) ,
    (λ z₁ → σₑ z₁ , tₑ z₁) ,
    (λ γₛ z₁ → σ₀ γₛ z₁ , t₀ γₛ z₁) ,
    (λ γₛ γₑ γ z₁ → σ₁ γₛ γₑ γ z₁ , t₁ γₛ γₑ γ z₁)
fam-c ._,_ {i = z} (σₛ , σₑ , σ₀ , σ₁) (tₛ , t₀)
  = (λ z₁ → σₛ z₁ , tₛ z₁) , σₑ , (λ γₛ z₁ → σ₀ γₛ z₁ , t₀ γₛ z₁) , σ₁
fam-c .,∘ {i = z} = refl
fam-c .,∘ {i = ω} = refl
fam-c .p,q {i = z} = refl
fam-c .p,q {i = ω} = refl
fam-c .p∘, {i = z} = refl
fam-c .p∘, {i = ω} = refl
fam-c .q[,] {i = z} = refl
fam-c .q[,] {i = ω} = refl

(fam-c [ t ]#) σ γ = t (σ .proj₂ .proj₁ γ)
fam-c .[id]# = refl
fam-c .[∘]# = refl
(fam-c ▷#) (Γₛ , Γₑ , Γ₀ , Γ₁) = Γₛ , ⊥ , Γ₀ , λ _ _ _ → ⊥
fam-c .p# = (λ z₁ → z₁) , (λ ()) , (λ γₛ z₁ → z₁) , λ γ ()
fam-c .q# γ = γ
(fam-c ,# σ) π = σ .proj₁ , π , σ .proj₂ .proj₂ .proj₁ , (λ γₛ γₑ γ z₁ → π γₑ)
fam-c .,#∘ = refl
fam-c .p,#q = refl
fam-c .p∘,# {π = π} =
  Σ≡ refl (Σ≡ (funext (λ γₑ → exfalso-empty-prop (π γₑ)))
    (Σ≡ refl ( funext (λ γₛ → funext (λ γₑ → exfalso-empty-prop (π γₑ))))))
fam-c .q[,#] = refl
fam-c .↓ (xₛ , xₑ , x₀ , x₁) = xₛ , x₀
fam-c .↓[] = refl
fam-c .↑ (xₛ , x₀) = xₛ , (λ ()) , x₀ , λ γₛ ()
fam-c .↑↓ {t = t} = Σ≡ refl (Σ≡ (funext (λ ()))
  (Σ≡ refl (funext (λ x → funext (λ y → exfalso-empty-prop y)))))
fam-c .↓↑ = refl

open Π-structure

fam-Π : Π-structure fam-s fam-c
fam-Π .Π ω (Aₛ , A₀ , A₁) (Bₛ , B₀ , B₁) =
  (λ γ → (a : Aₛ γ) → Bₛ (γ , a))
  , (λ γₛ γ fₛ → ∀ aₛ → (a : A₀ γₛ γ aₛ) → B₀ (γₛ , aₛ) (γ , a) (fₛ aₛ))
  , λ γₛ γ fₛ fₑ f₀
    → ∀ aₛ aₑ a → A₁ γₛ γ aₛ aₑ a → B₁ (γₛ , aₛ) (γ , a) (fₛ aₛ) (fₑ @ aₑ) (f₀ aₛ a)
fam-Π .Π z (Aₛ , A₀ , A₁) (Bₛ , B₀ , B₁) =
  (λ γ → (a : Aₛ γ) → Bₛ (γ , a))
  , (λ γₛ γ fₛ → ∀ aₛ → (a : A₀ γₛ γ aₛ) → B₀ (γₛ , aₛ) (γ , a) (fₛ aₛ))
  , λ γₛ γ fₛ fₑ f₀
    → ∀ aₛ a → B₁ (γₛ , aₛ) (γ , a) (fₛ aₛ) fₑ (f₀ aₛ a)
fam-Π .Π[] {i = ω} = refl
fam-Π .Π[] {i = z} = refl
fam-Π .lam {i = ω} (fₛ , fₑ , f₀ , f₁) =
  (λ γ a → fₛ (γ , a))
  , (λ γₑ → ƛ (λ x → fₑ (γₑ , x)))
  , (λ γₛ γ aₛ a → f₀ (γₛ , aₛ) (γ , a))
  , λ γₛ γₑ γ γ' aₛ aₑ a a' → f₁ (γₛ , aₛ) (γₑ , aₑ) (γ , a) (γ' , a')
fam-Π .lam {i = z} (fₛ , fₑ , f₀ , f₁) =
  (λ γ a → fₛ (γ , a))
  , fₑ
  , (λ γₛ γ aₛ a → f₀ (γₛ , aₛ) (γ , a))
  , λ γₛ γₑ γ γ' aₛ a → f₁ (γₛ , aₛ) γₑ (γ , a) γ'
fam-Π .lam[] {i = z} = refl
fam-Π .lam[] {i = ω} = refl
fam-Π .ap {i = z} (tₛ , tₑ , t₀ , t₁) =
  (λ (γ , a) → tₛ γ a)
  , tₑ
  , (λ (γₛ , aₛ) (γ , a) → t₀ γₛ γ aₛ a)
  , λ (γₛ , aₛ) γₑ (γ , a) γ' → t₁ γₛ γₑ γ γ' aₛ a
fam-Π .ap {i = ω} (tₛ , tₑ , t₀ , t₁) =
  (λ (γ , a) → tₛ γ a)
  , (λ (γₑ , aₑ) → tₑ γₑ @ aₑ)
  , (λ (γₛ , aₛ) (γ , a) → t₀ γₛ γ aₛ a)
  , λ (γₛ , aₛ) (γₑ , aₑ) (γ , a) (γ' , a') → t₁ γₛ γₑ γ γ' aₛ aₑ a a'
fam-Π .Πβ {i = z} = refl
fam-Π .Πβ {i = ω} = refl
fam-Π .Πη {i = z} = refl
fam-Π .Πη {i = ω} = refl
```

```
    open U-structure

    fam-U : U-structure fam-s fam-c
    fam-U .U =
      (λ γ → Set)
      , (λ γₛ γ Aₛ → Σ[ A₀ ∈ (Aₛ → Set) ] ((aₛ : Aₛ) → Λ → A₀ aₛ → Set))
      , λ γ γₛ aₛ aₑ a₀ → ⊤
    fam-U .U[] = refl
    fam-U .El (tₛ , t₀) =
      tₛ , (λ γₛ γ aₛ → t₀ γₛ γ .proj₁ aₛ) , λ γₛ γ aₛ aₑ aₒ → t₀ γₛ γ .proj₂ aₛ aₑ aₒ
    fam-U .El[] = refl
    fam-U .code (Aₛ , A₀ , A₁) = Aₛ , λ γₛ γ → A₀ γₛ γ , A₁ γₛ γ
    fam-U .code[] = refl
    fam-U .El-code = refl
    fam-U .code-El = refl

    fam : CwFwE
    fam .CwFwE.sorts = fam-s
    fam .CwFwE.core = fam-c
    fam .CwFwE.Π-str = fam-Π
    fam .CwFwE.U-str = fam-U

opaque
  unfolding fam

  open CwFwE-sorts fam-s
  open in-CwFwE-sorts fam-s
  open CwFwE-core fam-c
  open in-CwFwE-core fam-c
  open Π-structure fam-Π
  open U-structure fam-U

  Nat : Ty Γ
  Nat =
    (λ γ → ℕ)
    , (λ γₛ z₁ z₂ → ⊤)
    , λ γₛ γ aₛ aₑ a₀ → (aₑ ≡ embed-nat aₛ) true

  zero : Tm Γ ω (Nat {Γ})
  zero =
    (λ γ → 0)
    , (λ _ → zeroΛ)
    , (λ γₛ γ → tt)
    , (λ γₛ γₑ γ γ' → by refl)

  succ : Tm Γ ω (Nat {Γ}) → Tm Γ ω (Nat {Γ})
  succ (nₛ , nₑ , n₀ , n₁) =
    (λ γ → suc (nₛ γ))
    , (λ γₑ → succΛ (nₑ γₑ) )
    , (λ γₛ γ → tt)
    , λ γₛ γₑ γ γ' → by (cong succΛ (n₁ γₛ γₑ γ γ' .witness))

  tracking : Tm • ω (Π {•} ω (Nat {•}) (Nat {• ▷[ z ] (Nat {•})}))
    → Σ[ fₛ ∈ (ℕ → ℕ) ]
      Σ[ fₑ ∈ Λ ]
      (∀ aₛ aₑ → (aₑ ≡ embed-nat aₛ) true → ((fₑ @ aₑ) ≡ embed-nat (fₛ aₛ)) true)
  tracking (fₛ , fₑ , f₀ , f₁) = fₛ tt , fₑ tt , λ aₛ aₑ → f₁ tt tt tt tt aₛ aₑ tt

  noninterference : Tm • ω (Π {•} z (Nat {•}) (Nat {• ▷[ z ] (Nat {•})}))
    → Σ[ fₛ ∈ (ℕ → ℕ) ]
      Σ[ fₑ ∈ Λ ]
      Σ[ xₛ ∈ ℕ ]
      ((∀ aₛ → (fₛ aₛ ≡ xₛ) true) × (fₑ ≡ embed-nat xₛ) true)
  noninterference (fₛ , fₑ , f₀ , f₁) =
    fₛ tt , fₑ tt , fₛ tt 0
    , (λ n →
      let by pn = f₁ tt tt tt tt n tt in
      let by p0 = f₁ tt tt tt tt 0 tt in by (embed-nat-inj _ _ (trans (sym pn) p0)))
    , f₁ tt tt tt tt 0 tt
```

References

[1] Andreas Abel, Nils Anders Danielsson, and Andrea Vezzosi. Compiling programs with erased univalence, 2022. URL: https://www.cse.chalmers.se/~nad/publications/abel-danielsson-vezzosi-erased-univalence.pdf.

[2] Andreas Abel and Brigitte Pientka. Higher-order dynamic pattern unification for dependent types and records. *Lect. Notes Comput. Sci.*, 6690 LNCS:10–26, 2011. URL: https://link.springer.com/chapter/10.1007/978-3-642-21691-6_5.

[3] Run-time irrelevance — Agda 2.9.0 documentation. URL: https://agda.readthedocs.io/en/latest/language/runtime-irrelevance.html.

[4] Thorsten Altenkirch, Paolo Capriotti, Gabe Dijkstra, Nicolai Kraus, and Fredrik Nordvall Forsberg. Quotient inductive-inductive types. In *Foundations of Software Science and Computation Structures*, pages 293–310. Springer International Publishing, 2018. URL: http://dx.doi.org/10.1007/978-3-319-89366-2_16.

[5] Thorsten Altenkirch and Ambrus Kaposi. Type theory in type theory using quotient inductive types. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '16, pages 18–29, New York, NY, USA, 2016. Association for Computing Machinery. URL: https://doi.org/10.1145/2837614.2837638.

[6] Danil Annenkov, Paolo Capriotti, Nicolai Kraus, and Christian Sattler. Two-level type theory and applications. *Math. Struct. Comput. Sci.*, 33:688–743, 2023. URL: https://www.cambridge.org/core/services/aop-cambridge-core/content/view/4914DB4F8E8305DFC68F9CDCA9D0C8D0/S0960129523000130a.pdf/div-class-title-two-level-type-theory-and-applications-div.pdf.

[7] Robert Atkey. Syntax and semantics of quantitative type theory. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science*, LICS '18, pages 56–65, New York, NY, USA, 2018. Association for Computing Machinery. URL: https://doi.org/10.1145/3209108.3209189.

[8] Rafaël Bocquet. *Relative induction principles for second-order generalized algebraic theories*. PhD thesis, Eötvös Loránd University, 2025. URL: https://rafaelbocquet.gitlab.io/pdfs/thesis.pdf.

[9] Rafaël Bocquet, Ambrus Kaposi, and Christian Sattler. For the metatheory of type theory, internal sconing is enough, 2023. URL: http://dx.doi.org/10.4230/LIPIcs.FSCD.2023.18.

[10] Edwin Brady, Conor McBride, and James McKinna. Inductive families need not store their indices. In *Types for Proofs and Programs*, pages 115–129. Springer Berlin Heidelberg, 2004. URL: http://dx.doi.org/10.1007/978-3-540-24849-1_8.

[11] L Cardelli. Phase distinctions in type theory. Unpublished manuscript, 1988. URL: http://lucacardelli.name/Papers/PhaseDistinctions.A4.pdf.

[12] John Cartmell. Generalised algebraic theories and contextual categories. *Ann. Pure Appl. Logic*, 32:209–243, 1986. URL: http://dx.doi.org/10.1016/0168-0072(86)90053-9.

[13] Simon Castellan, Pierre Clairambault, and Peter Dybjer. Categories with families: Unityped, simply typed, and dependently typed. *arXiv [cs.LO]*, 2019. URL: http://arxiv.org/abs/1904.00827.

[14] Jesper Cockx. agda/agda: Should erasure violations be hard errors more often? # 5703, 2021. URL: https://github.com/agda/agda/issues/5703.

[15] Nils Anders Danielsson. Logical properties of a modality for erasure. Technical report, University of Gothenburg, 2020. URL: https://www.cse.chalmers.se/~nad/publications/danielsson-erased.pdf.

[16] Steve Dunham. idris-lang/Idris2: Holes at unerased quantity produce bad code # 3367, 2024. URL: https://github.com/idris-lang/Idris2/issues/3367.

[17] Naïm Camille Favier. ErasureOpen. URL: https://agda.monade.li/ErasureOpen.

[18] Daniel Gratzer, Michael Shulman, and Jonathan Sterling. Strict universes for grothendieck topoi. *arXiv [math.CT]*, 2022. URL: https://share.google/2bfRTY63iMIET0oJg.

[19] Harrison Grodin, Runming Li, and Robert Harper. Abstraction functions as types. *arXiv [cs.PL]*, 2025. URL: http://dx.doi.org/10.48550/arXiv.2502.20496.

[20] Adam Gundry and Conor Mcbride. Phase your erasure, 2013. URL: https://personal.cis.strath.ac.uk/conor.mcbride/pub/phtt.pdf.

[21] Robert Harper, John C Mitchell, and Eugenio Moggi. Higher-order modules and the phase distinction. In *Proceedings of the 17th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '90*, pages 341–354, New York, New York, USA, 1990. ACM Press. URL: http://dx.doi.org/10.1145/96709.96744.

[22] Martin Hofmann. *Syntax and semantics of dependent types*, pages 79–130. Cambridge University Press, Cambridge, 1997. URL: http://dx.doi.org/10.1017/CBO9780511526619.004.

[23] Martin Hofmann and Thomas Streicher. Lifting grothendieck universes. Unpublished note, 1997.

[24] Jasper Hugunin. Why not W?, 2021. URL: http://dx.doi.org/10.4230/LIPIcs.TYPES.2020.8.

[25] Multiplicities — Idris2 0.0 documentation. URL: https://idris2.readthedocs.io/en/latest/tutorial/multiplicities.html#multiplicities.

[26] Bernardy Jean-Philippe and Guilhem Moulin. Type-theory in color. In *Proceedings of the 18th ACM SIGPLAN international conference on Functional programming*, New York, NY, USA, 2013. ACM. URL: http://dx.doi.org/10.1145/2500365.2500577.

[27] Ambrus Kaposi, Simon Huber, and Christian Sattler. Gluing for type theory, 2019. URL: http://dx.doi.org/10.4230/LIPIcs.FSCD.2019.25.

[28] Ambrus Kaposi and Szumi Xie. Second-order generalised algebraic theories: Signatures and first-order semantics, 2024. URL: http://dx.doi.org/10.4230/LIPIcs.FSCD.2024.10.

[29] András Kovács. 04-implicit-args at master · AndrasKovacs/elaboration-zoo. URL: https://github.com/AndrasKovacs/elaboration-zoo/tree/master/04-implicit-args.

[30] András Kovács. Staged compilation with two-level type theory. *Proc. ACM Program. Lang.*, 6:540–569, 2022. URL: https://dl.acm.org/doi/10.1145/3547641.

[31] András Kovács. *Type-theoretic signatures for algebraic theories and inductive types*. PhD thesis, Eötvös Loránd University, 2023. URL: https://andraskovacs.github.io/pdfs/phdthesis_compact.pdf.

[32] F William Lawvere. *Functorial semantics of algebraic theories*. PhD thesis, Columbia University, 1963. URL: http://www.tac.mta.ca/tac/reprints/articles/5/tr5.pdf.

[33] Daniel R Licata, Ian Orton, Andrew M Pitts, and Bas Spitters. Internal universes in models of homotopy type theory, 2018. URL: http://dx.doi.org/10.4230/LIPIcs.FSCD.2018.22.

[34] Conor McBride. *I Got Plenty o' Nuttin'*, pages 207–233. Lecture notes in computer science. Springer International Publishing, Cham, 2016. URL: http://dx.doi.org/10.1007/978-3-319-30936-1_12.

[35] Nathan Mishra-Linger and Tim Sheard. *Erasure and Polymorphism in Pure Type Systems*, pages 350–364. Springer Berlin Heidelberg, 2008. URL: http://dx.doi.org/10.1007/978-3-540-78499-9_25.

[36] Andreas Nuyts and Dominique Devriese. Transpension: The right adjoint to the pi-type. *arXiv [cs.LO]*, 2020. URL: http://dx.doi.org/10.48550/arXiv.2008.08533.

[37] Mitchell Riley. A type theory with a tiny object. *arXiv [math.CT]*, 2024. URL: http://arxiv.org/abs/2403.01939.

[38] Jon Sterling. On the relationship between QTT and STC, 2023. URL: https://www.jonmsterling.com/0094/.

[39] Jonathan Sterling. Algebraic type theory and universe hierarchies. *arXiv [cs.LO]*, 2019. URL: http://arxiv.org/abs/1902.08848.

[40] Jonathan Sterling. First steps in synthetic tait computability: The objective metatheory of cubical type theory, 2022. URL: http://dx.doi.org/10.1184/R1/19632681.v1.

[41] Jonathan Sterling and Carlo Angiuli. Normalization for cubical type theory. In *2021 36th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*, pages 1–15. IEEE, 2021. URL: http://dx.doi.org/10.1109/LICS52264.2021.9470719.

[42] Jonathan Sterling and Robert Harper. Logical relations as types: Proof-relevant parametricity for program modules. *J. ACM*, 68:1–47, 2021. URL: http://dx.doi.org/10.1145/3474834.

[43] Thomas Streicher. *UNIVERSES IN TOPOSES*, pages 78–90. Oxford University PressOxford, 2005. URL: http://dx.doi.org/10.1093/acprof:oso/9780198566519.003.0005.

[44] Matúš Tejiščák. Erasure in dependently typed programming, 2020. URL: http://dx.doi.org/10.17630/STA/677.

[45] Taichi Uemura. *Abstract and concrete type theories*. PhD thesis, University of Amsterdam, 2021. URL: https://pure.uva.nl/ws/files/62028111/Thesis.pdf.

[46] Théo Winterhalter. Dependent ghosts have a reflection for free. *Proc. ACM Program. Lang.*, 8:630–658, 2024. URL: https://hal.science/hal-04163836/file/icfp24main-p74-p-dba0e9df86-78716-final.pdf.

[47] Théo Winterhalter, Johann Rosain, and Matthieu Sozeau. A ghost sort for proof-relevant yet erased data in rocq and MetaRocq . *TYPES 2025 Abstract*, 2025. URL: https://msp.cis.strath.ac.uk/types2025/abstracts/TYPES2025_paper81.pdf.